Graph Prompt Learning: A Comprehensive Survey and Beyond

Abstract—Artificial General Intelligence (AGI) has accelerated foundation-model research across multiple domains, yet graph data remains less explored than text and vision in this paradigm. This survey reviews graph prompt learning from the perspectives of cross-modality, cross-domain, and cross-task generalization, and develops a unified view based on prompt tokens, token structures, and insertion patterns. It further examines the relationship between graph pre-training and prompting, discusses how prompts align downstream tasks with pretext objectives, and summarizes representative prompt tuning strategies. The resulting taxonomy covers over one hundred representative studies in recent years across pre-training strategies, prompt design, multi-modal prompting, and graph domain adaptation. We also summarize application scenarios and supporting resources, including ProG and a curated repository of papers, benchmarks, and code. Finally, the survey discusses current challenges and future directions for graph prompting in graph representation learning and related AGI-oriented applications. Relevant resources are available at https://github.com/WxxShirley/Awesome-Graph-Prompt and https://github.com/sheldonresearch/ProG.

Index Terms—graph prompt, graph pre-training, graph learning, artificial general intelligence.

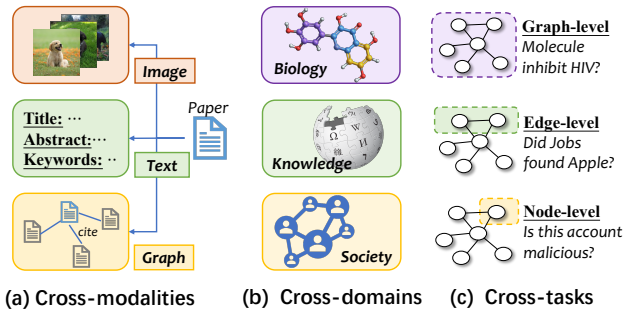


Fig. 1: Three recurring challenges for more general graph learning systems: (a) cross-modal transfer, (b) cross-domain transfer, and (c) cross-task transfer.

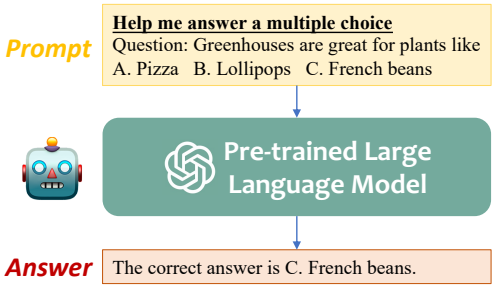
1 Introduction

Recent advances in foundation models have substantially reshaped research and applications in Natural Language Processing (NLP) and Computer Vision (CV). In contrast, the development of similarly mature adaptation paradigms for graph-structured data remains comparatively limited [10, 2, 53, 97, 124]. This gap is important because graphs are a natural abstraction for relational and structured data, and they play a central role in applications such as molecular discovery, drug design, recommender systems, and scientific modeling [52, 130, 116, 73, 69, 96]. As a result, there is growing interest in general-purpose graph learning systems that can better transfer across tasks, domains, and modalities.

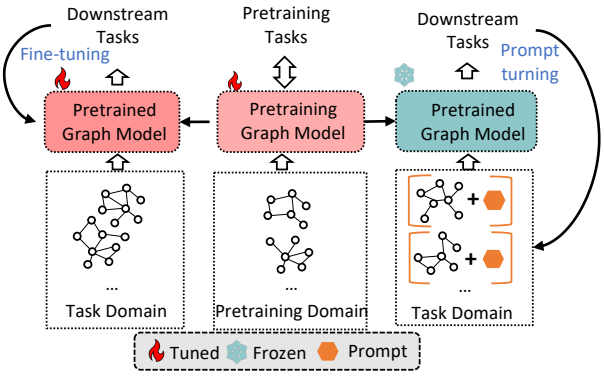
However, achieving this goal remains difficult. Figure 1 highlights three recurring technical challenges for graph learning research. ***(1) How can models generalize across modalities?*** Recent multimodal foundation models have shown strong capabilities in aligning and reasoning across textual and visual inputs [10, 124, 2]. In graph-centered settings, however, comparable progress remains much more limited, especially when graph structure must be integrated with textual, visual, or scientific signals [48]. ***(2) How can models generalize across domains?*** Transfer learning has been effective in many machine learning settings, but transferring knowledge between graph domains remains challenging because both semantic spaces and structural patterns may shift substantially [137, 133]. This makes graph domain adaptation a difficult and still open research problem. ***(3) How can models generalize across tasks?*** A large portion of the existing literature focuses on transferring pre-trained graph knowledge across tasks within the same graph domain, such as node classification, link prediction, and graph classification [84, 56, 86, 13, 136, 35, 79, 20]. Yet, compared with NLP and CV, standardized real-world evaluations and deployment-oriented studies for graph prompting remain relatively limited. Taken together, these observations suggest that graph prompting should be studied not only as a task adaptation trick, but also as part of a broader effort toward more general and reusable graph learning systems.

These challenges have made parameter-efficient adaptation strategies increasingly attractive [18, 45]. Among them, prompt learning has emerged as a practical mechanism for adapting pre-trained models without repeatedly updating all model parameters [70, 92, 53]. Prompting modifies the input or task formulation so that downstream problems better align with behaviors induced during pre-training. In many settings, this enables practitioners to reuse pre-trained models with substantially fewer trainable parameters than full fine-tuning [2, 39, 45, 78]. For graph learning, this property makes prompting a promising, though not universally sufficient, mechanism for task adaptation, cross-domain transfer, and multimodal interaction.

From a representational perspective, several data modalities can be reinterpreted through graph-structured abstractions. A sentence may be modeled as a dependency graph or token interaction graph, and an image may be represented as a grid graph or region graph. This perspective motivates the adaptation of successful prompting ideas from language



(a) language prompt



(b) graph prompt v.s. graph fine-tune

Fig. 2: Language prompt v.s. Graph prompt. A textual prompt is added to the original input to let the pre-trained large language model perform a new task (e.g. the multi-choice question-answering). Inspired by the language prompt, a graph prompt can be also used with graphs in the same way.

and vision to graph learning [84, 56, 86, 13, 136, 60, 23, 4, 21].

Although graph prompting is partly inspired by prompting techniques in NLP, the two are not directly analogous [86]. **First**, the design space of graph prompts is substantially richer. In language models, prompts may take the form of discrete templates, soft prompt vectors, prefixes, or instruction-style conditioning [2, 18]. In graph learning, however, a prompt often also involves structural design choices, insertion mechanisms, and answering functions, rather than prompt content alone. **Second**, aligning downstream graph problems with pre-training objectives is typically more difficult than in language tasks [56, 86]. A typical language pre-training task such as masked word prediction [10] shares a relatively direct interface with many downstream tasks [53], whereas graph tasks span node-level [22], edge-level [127], and graph-level objectives [82, 85], making pre-training pretexts less directly reusable. **Third**, graph prompts are usually less interpretable to non-specialists than natural-language prompts, because their effects are mediated by graph topology, feature transformations, and message passing. These differences make it necessary to examine graph prompting on its own terms rather than treating it as a straightforward extension of NLP prompting.

The literature on graph prompting has expanded rapidly in recent years, but it remains fragmented across task settings, graph modalities, and adaptation strategies. Existing works often emphasize different perspectives, making direct comparison difficult and slowing the systematic accumulation of knowledge in this area. This survey therefore provides a structured review of graph prompting through the lens of the three transfer challenges discussed above, while also clarifying its relationship to graph pre-training, graph foundation models, and multimodal graph learning. To organize the field, we consider the following research questions (RQs):

- **RQ1: How can existing graph prompting methods be organized under a unified analytical framework?** We seek a common vocabulary for comparing methods that differ in prompt representation, insertion mechanism, and adaptation objective.
- **RQ2: What functional role do prompts play in graph representation learning?** We study what graph prompts actually contribute, how they interact with graph learners, and why they may behave differently from conventional fine-tuning.
- **RQ3: What are the main design dimensions of graph prompts and prompt tuning strategies?** We examine how graph prompts are constructed, how they align downstream tasks with pre-training objectives, and how they are optimized in practice.
- **RQ4: How can graph prompting be supported in practical applications and software ecosystems?** We discuss the current state of toolkits, benchmarks, and reusable implementations for graph prompting in real-world scenarios.
- **RQ5: What limitations remain in current graph prompting research, and which future directions appear most promising?** We summarize open challenges and outline research directions that may shape the next stage of the field.

RQ1 motivates a unified framework for analyzing graph prompt learning. This framework decomposes graph prompts into prompt tokens, token structures, and inserting patterns, thereby providing a stable lens for comparing existing variants.

RQ2 concerns the relationship between prompts and existing graph models through the lenses of flexibility and expressiveness, leading to a deeper understanding of the role of graph prompts. Unlike many prompt learning surveys in NLP [53], which treat prompting primarily as a mechanism for bridging pre-training and downstream tasks, we discuss graph prompting as being intertwined with the modeling capacity of graph learners themselves. This perspective helps clarify why prompting is promising in the graph domain and how it differs from conventional fine-tuning methods [34].

RQ3 motivates a taxonomy that organizes representative graph prompting studies according to node-level, edge-level, and graph-level settings, while aligning them with the broader context of pre-training objectives. This organization clarifies prompt mechanisms within the overall "pre-training and prompting" workflow.

RQ4 concerns the supporting ecosystem for graph prompting. We discuss toolkits such as ProG (prompt graph) [1] and curated repositories[2] that improve accessibility, reproducibility, and comparability for researchers and practitioners working on graph prompting.

Beyond these questions, the survey further discusses application scenarios, current challenges, and future research

1. https://github.com/sheldonresearch/ProG
2. https://github.com/WxxShirley/Awesome-Graph-Prompt

directions, thereby outlining a broader roadmap for the development of graph prompt learning (RQ5). The contributions are summarised as follows:

- **Enabling Comprehensive Analysis.** We develop a unified framework for analyzing graph prompt learning, providing a structured view of prompt tokens, token structures, and inserting patterns.
- **Prompt–Model Perspective.** We discuss graph prompts not only as task adapters but also as mechanisms that interact with the flexibility and expressiveness of graph learners.
- **A Systematic Taxonomy of Graph Prompting.** We synthesize representative graph prompting studies and organize them within the overarching "pre-training and prompting" workflow.
- **Empowering the Graph Prompting Ecosystem.** We summarize toolkits and community resources that support reproducible graph prompting research.
- **Charting a Path Forward.** We provide a detailed discussion of current challenges and future directions in the field.

Roadmap. The remainder of this survey is organized as follows: Section 2 presents the survey methodology, followed by preliminaries in Section 3, pre-training methods in Section 4, and prompting methods for graph models in Section 5. Section 7 discusses potential applications of graph prompting, and Section 8 presents ProG. Section 9 discusses current challenges and future directions. Section 10 concludes the survey and presents the contribution declaration of the authors.

2 Survey Methodology

2.1 Research Objectives

This survey studies graph prompting as a route toward more transferable and reusable graph learning systems. We organize the discussion around the three transfer challenges summarized in Table 1: transfer across modalities, domains, and tasks. Prompting has become an important adaptation mechanism in NLP and vision [2, 18, 97], but its role in graph learning is still less settled. Our goal is therefore not to assume that prompting is a universal solution, but to examine when and how graph prompts help graph models transfer across tasks and domains, and how they interact with other modalities such as text and images. Beyond these shared transfer challenges, graph prompting also differs from its counterparts in NLP and CV, which motivates the more specific research questions listed in Table 1.

TABLE 1: Research Questions and Scope

Transfer Challenges in Graph Learning	P1: How to Make the Model General for Different Modalities? (Section 6.1)
	P2: How to Make the Model General for Different Domains? (Section 6.2)
	P3: How to Make the Model General for Different Tasks? (Section 5)
Detailed Questions of Graph Prompt	Q1: How to Understand Existing Work with a Unified Framework? (Section 5.1)
	Q2: What's the Nature of Graph Prompt? (Section 3.5, Section 5.4)
	Q3: How to Design Graph Prompts? (Section 5)
	Q4: How to Deploy Graph Prompts in Real-world Applications? (Section 7, Section 8)
	Q5: What Are the Current Challenges and Future Directions? (Section 9.1)

2.2 Taxonomy

Figure 3 summarizes how this survey organizes the graph prompting literature. The taxonomy is designed to show how prompt-based methods relate to pre-training objectives, transfer settings, and application scenarios in graph learning.

(1) Pre-training Strategies for Graph Prompt.

Prompting methods are often motivated by the need to align downstream tasks with behaviors induced during pre-training. For this reason, the survey first reviews representative pre-training strategies in graph learning before turning to prompting itself. We group existing pre-training approaches into node-level, edge-level, graph-level, and multi-task settings, and then examine how later prompting methods reuse or adapt these objectives. This setup makes it easier to compare prompting methods against the pre-training assumptions on which they depend.

(2) Prompting Methods in Graph Learning.

To cover the three transfer challenges defined in Table 1, we analyze graph prompting from three directions: i. prompt design for graph tasks (Section 5); ii. multi-modal prompting with graphs (Section 6); and iii. graph domain adaptation with prompting techniques (Section 6.2). Within each direction, we revisit the five detailed research questions in Table 1 and use them to connect method design, adaptation mechanism, and practical scope.

i. Prompt Design for Graph Tasks. In this part, we introduce a unified analytical framework for graph prompt design. We organize existing methods around three components: *prompt tokens*, which represent prompt content; *token structures*, which specify how multiple prompt units are organized; and *inserting patterns*, which determine how prompts are combined with the original graph. We further discuss answering functions, namely how downstream predictions are produced from prompted graphs, and summarize representative optimization strategies such as meta-learning, task-specific tuning, and pretext-aligned tuning. This organization lets us compare methods in terms of shared assumptions, design differences, and trade-offs.

ii. Multi-modal Prompting with Graphs. In this part, we discuss prompting methods for text-attributed and other multi-modal graph settings. The recent growth of graph-LLM and graph-language research has expanded this area substantially, so our goal here is not to provide a full survey of graph-LLM methods. Instead, we focus on prompting-oriented interfaces and adaptation mechanisms that explicitly involve graph structure.

iii. Graph Domain Adaptation through Prompting Techniques. In this part, we review two lines of work: methods that address semantic alignment across graph domains, and methods that address structural alignment.

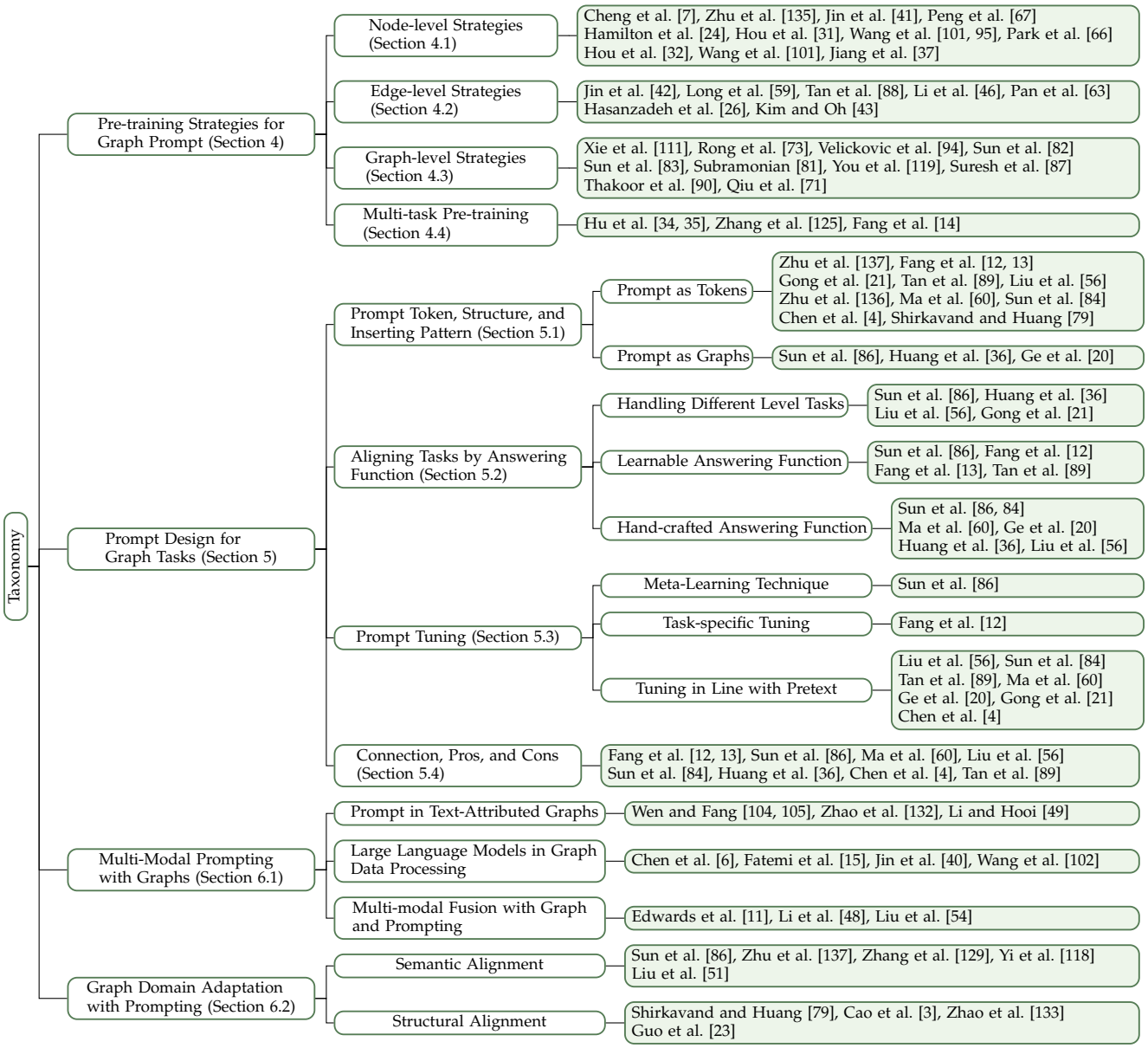


Fig. 3: Taxonomy of this Survey with Representative Works.

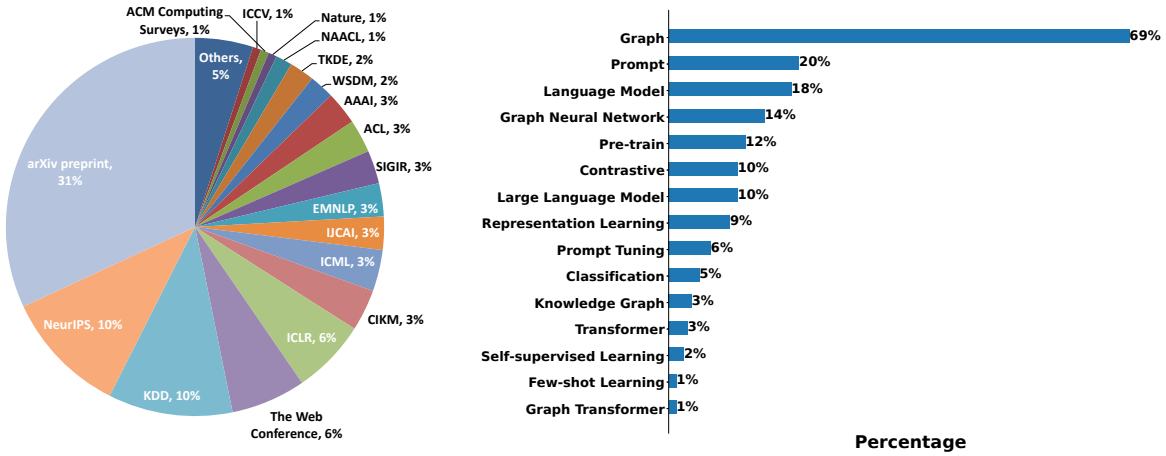
2.3 Literature Overview

This survey covers representative graph prompting papers published through early 2026, with emphasis on conference and journal articles as well as influential arXiv preprints. The surveyed literature includes work from venues such as NeurIPS, SIGKDD, The Web Conference, ICLR, CIKM, ICML, IJCAI, EMNLP, SIGIR, ACL, AAAI, WSDM, and TKDE. We also include recent preprints when they had visible influence on subsequent literature, benchmarks, or public implementations before formal publication. Figure 4a summarizes the venue distribution of the collected papers, and Figure 4b shows the most frequent keywords in their titles.

2.4 Connection to Existing Work

Recent surveys on graph prompt learning differ mainly in scope and organizing principle. Wu et al. [109] reviewed the early wave of graph prompting methods with an emphasis on techniques, applications, and open challenges when the methodological space was still relatively compact. Long et al. [58] expanded the discussion to design principles, application scenarios, benchmark issues, and prompt tuning strategies. Fu et al. [17] organized the literature around the "pre-train and prompt" workflow and emphasized data-level, representation-level, and task-level perspectives.

Our emphasis differs in two respects. First, we adopt a unified decomposition of graph prompting into prompt tokens, token structures, insertion patterns, answering functions, and prompt tuning strategies. Second, we place graph prompting within a broader taxonomy that also includes pre-training strategies, multi-modal prompting, and graph domain adaptation. We further retain a practical perspective through the discussion of application scenarios and supporting toolkits such as ProG. Related surveys on graph foundation models, graph-LLM integration, and graph



(a) Venue distribution of collected papers.

(b) Top 15 Keywords appeared in titles of collected papers

Fig. 4: Statistics of the collected papers

self-supervised learning [52, 50, 112, 121] provide broader background, whereas this survey concentrates specifically on graph prompting and its surrounding adaptation mechanisms.

3 Preliminaries

This section introduces the background needed for the rest of the survey. We first define the basic notation used throughout the paper, then review the main stages of graph representation learning, including shallow embeddings, supervised GNNs, and pre-training-based transfer. We also briefly revisit the development of prompt learning and explain why prompting deserves separate discussion in graph learning rather than being treated as a direct copy of NLP practice.

3.1 Notations

Let a graph be denoted by $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, v_2, \ldots, v_N\}$ is the node set and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the edge set. Each node v_i is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^D$. We use $\mathbf{X} \in \mathbb{R}^{N \times D}$ to denote the node feature matrix and $\mathbf{A} \in \{0,1\}^{N \times N}$ to denote the adjacency matrix, where $\mathbf{A}_{ij} = 1$ if and only if $(v_i, v_j) \in \mathcal{E}$.

3.2 Graph Representation Learning

Graph representation learning has developed through several overlapping stages. Early work focused on shallow embedding methods, which map nodes to learnable low-dimensional vectors and are then reused in downstream tasks. Representative examples include DeepWalk [68] and node2vec [22]. These methods are flexible because the learned embeddings can often be reused across multiple tasks, but they are usually transductive and do not naturally incorporate rich node attributes.

Shallow embedding methods are often grouped by how they define node similarity. One common line is factorization-based [62, 126]; another is based on random walks [22, 68, 100, 99]. These methods remain useful when the goal is to preserve structural proximity in a compact representation, but their transfer behavior is limited once node attributes, unseen nodes, or task-specific supervision become central.

Graph neural networks (GNNs) address some of these limitations by learning task-relevant representations through parameterized neighborhood aggregation. Most GNNs follow a message-passing framework and use a more expressive encoder than shallow embeddings. Representative architectures include spectral and spatial variants [9, 29, 61, 24, 93, 77, 114]. In many settings, GNNs outperform shallow embeddings because they can combine graph structure with node features. The trade-off is that they are usually trained for specific tasks and often require supervision or repeated re-training when the task changes.

This contrast motivates the transfer problem that runs through the rest of the survey. Shallow embeddings are flexible but limited in modeling power. Task-specific GNNs are more expressive but less reusable across settings. Pre-training and prompting can both be understood as attempts to improve transfer, albeit through different mechanisms.

3.3 Pre-training and Fine-tuning

Pre-training and fine-tuning has become a common transfer paradigm in graph representation learning. The usual workflow is to pre-train a model on large-scale graph data, with or without labels, and then adapt it to downstream tasks by updating model parameters. In graph learning, representative pre-training objectives include graph autoencoding [95], masked component modeling [34, 73], and graph contrastive learning [94, 82]. We review these objectives in detail in Section 4.

This paradigm improves transfer because the downstream model no longer starts from scratch. Still, fine-tuning remains model-centric: the usual way to adapt to a new task is to update some or all of the model parameters. This can work well, but it also means that transfer often depends on repeated optimization, access to supervision, and a stable match between the new task and the pre-trained model.

3.4 A Brief History of Prompt Learning

Prompt learning emerged as another way to adapt pre-trained models, especially when full fine-tuning became increasingly expensive. Rather than updating the whole model for every downstream problem, prompting changes

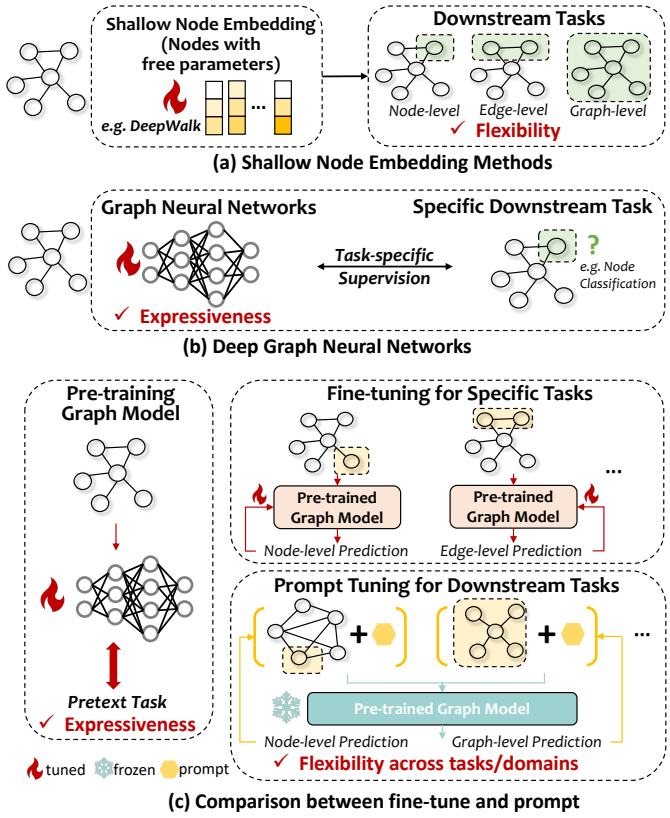


Fig. 5: A schematic view of flexibility and expressiveness in graph learning. (a) Shallow node embedding methods are often reusable across tasks but have limited modeling capacity. (b) Task-specific GNNs are more expressive but usually require re-training or supervision for new tasks. (c) Prompting can improve the trade-off by adapting the input while reusing a pre-trained graph model.

the input, the task interface, or a small set of learnable prompt parameters so that the downstream problem is presented in a form closer to what the model has already seen during pre-training [53].

In NLP, prompts take several forms. Some are discrete templates such as cloze-style prompts. Others are learned continuous prompts, including prefix-based methods [45, 47]. Prompt construction may rely on manual templates [2, 75, 76], discrete prompt search [39, 28, 78, 18], or prompting directly in the embedding space [47, 45, 92, 70]. These developments do not replace fine-tuning altogether. Instead, they expand the set of available adaptation mechanisms.

For graph learning, prompting is attractive for a similar reason: it may enable transfer with fewer updated parameters and with a clearer separation between the pre-trained graph encoder and the downstream adaptation module. At the same time, graph prompting is not a direct replica of NLP prompting, because graph inputs are structured objects and prompt design must also account for topology, insertion patterns, and graph-level prediction mechanisms.

3.5 Why Graph Prompt?

A. Differences between Graph Prompting and Graph Fine-tuning. The difference between graph prompting and graph fine-tuning is easiest to state in terms of where adaptation happens. Fine-tuning is primarily a model-level adaptation strategy: a pre-trained graph model is adjusted so that its parameters better fit a downstream task. Prompting, by contrast, is primarily an input-level or data-level adaptation strategy: the downstream graph is modified through prompt tokens, prompt structures, or prompt-induced interfaces before it is processed by a pre-trained model. In practice, graph prompting may still involve a small number of trainable parameters, but the central idea is not to re-optimize the full encoder for each new task.

B. Prompts as Learnable Graph Modifications. One useful way to understand graph prompting is to view it as a learnable approximation to graph transformation. Let φ^* denote a pre-trained graph model, $\mathcal{G}$ the original graph, $\mathcal{G}_p^*$ a prompt graph or prompt module, and $\psi(\mathcal{G}, \mathcal{G}_p^*)$ the prompted graph produced by inserting the prompt into the input. Let $\mathbf{g}(\mathbf{A}, \mathbf{X})$ denote a target transformation of the original graph structure and node features. Then, at a conceptual level, graph prompting can be written as

$$\varphi^*\left(\psi(\mathcal{G}, \mathcal{G}_p^*)\right) = \varphi^*(\mathbf{g}(\mathbf{A}, \mathbf{X})) + O_{p\varphi}^*, \tag{1}$$

where $O_{p\varphi}^*$ is the residual discrepancy between the prompted input and the ideal transformation under the pre-trained model φ^*. This expression should not be read as a formal guarantee. It is a conceptual lens: prompts can be interpreted as a learnable way to adjust graph structure, node features, or task interfaces so that the downstream input better matches the behavior already captured by the pre-trained encoder.

C. Uses Beyond Task Adaptation. This perspective also explains why graph prompting may be useful beyond ordinary task transfer. If a prompt can act as a learnable graph modification, then it may be used to adjust the input before inference, for example by compensating for structural mismatch, introducing auxiliary prompt nodes, or aligning a graph with another modality. In principle, this opens the door to applications beyond standard downstream adaptation, including domain alignment, multi-modal interfaces, and robustness-oriented input adjustment. Whether these uses help in practice remains task-dependent, but the key point is that prompting changes the way adaptation is expressed: not only through model updates, but also through learnable modifications to the input graph itself.

In short, graph prompting is worth separate study because it changes the unit of adaptation. Instead of relying solely on repeated model re-training, it offers a way to reuse a pre-trained graph encoder while expressing downstream adaptation through prompt parameters, prompt structure, and prompted graph construction. That shift is exactly what makes graph prompting both flexible and technically distinct from conventional fine-tuning.

4 Pre-training GNNs for Graph Prompting

Pre-training remains one of the main transfer mechanisms underlying graph prompting. Before prompts can adapt a graph model to a new task, the pre-trained encoder must already capture useful structural, semantic, or cross-instance regularities. For this reason, it is helpful to review graph pre-training not as a separate historical thread, but as part of the broader adaptation pipeline discussed in this survey.

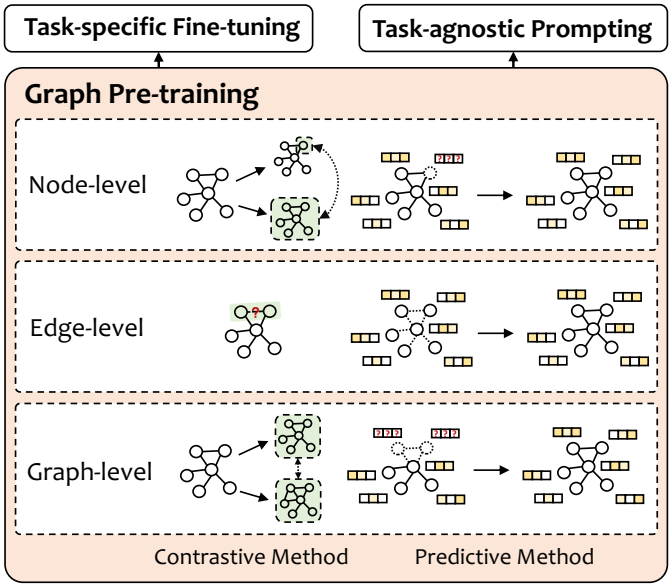


Fig. 6: Graph pre-training methods.

Figure 6 groups representative methods into four categories: node-level, edge-level, graph-level, and multi-task strategies. Representative recent surveys on self-supervised graph foundation models and graph foundation models more broadly provide useful background for this discussion [134, 103].

4.1 Node-level Strategies

Node-level pre-training methods aim to learn transferable local representations for nodes and their neighborhoods. As shown in Figure 6, these methods are usually built around either contrastive or predictive objectives. In contrastive settings, self-supervised signals are typically created by perturbing graph structure or node attributes and then maximizing agreement between the original and augmented views. Representative node-level contrastive methods include [7, 135, 41, 67, 37, 101].

Predictive node-level methods instead reconstruct masked or corrupted information from the unperturbed graph. Representative examples include masked autoencoding and related reconstruction objectives [31, 95, 66, 98, 32]. These methods can learn useful local patterns, but their transferability still depends on how well the pretext objective matches the downstream task. In particular, methods centered on local corruption or reconstruction may under-emphasize long-range dependencies or graph-level semantics. Recent work has also extended node-oriented pre-training to multi-graph settings, which is more consistent with the graph foundation model view of transfer [44].

4.2 Edge-level Strategies

Edge-level pre-training focuses on node interactions and pairwise relations, which makes it particularly relevant to tasks such as link prediction. One common approach is to discriminate whether an edge should exist between a pair of nodes, which can be viewed as a contrastive or relational discrimination objective [42, 59]. Another approach reconstructs masked or corrupted edges, often through adjacency recovery or related generative objectives [88, 46, 63, 26, 43].

These strategies are useful when downstream performance depends strongly on local connectivity patterns. Their limitation is that they are usually centered on structure alone. As a result, they may not transfer well when node attributes, higher-order semantics, or graph-level objectives matter more than pairwise edge recovery. Recent benchmark work likewise evaluates transfer more broadly than edge reconstruction alone [115].

4.3 Graph-level Strategies

Graph-level pre-training aims to improve whole-graph representations, especially for downstream tasks defined on subgraphs or complete graph instances. Similar to node- and edge-level settings, graph-level strategies are often divided into reconstruction-based and contrastive approaches. Reconstruction-based methods mask graph components and learn to recover them [111, 73]. Contrastive methods instead maximize mutual information between local and global views [94, 82, 27, 83, 81], or distinguish positive and negative graph pairs [119, 87, 90, 71].

These methods often produce useful graph-level representations, especially when the downstream objective is close to the pretext task. The main difficulty is transfer across tasks and datasets with larger semantic gaps. When the pre-training signal is too narrowly aligned with one kind of graph structure, the resulting encoder may transfer poorly and can even suffer from negative transfer [74]. More recent work has pushed graph-level pre-training toward cross-domain and text-attributed graph settings, which broadens the scope of transfer beyond a single graph family [30, 80].

4.4 Multi-task Pre-training

Multi-task pre-training combines several objectives so that the encoder learns from multiple views of graph data at once. Typical choices include joint optimization of attribute reconstruction and structural recovery [34, 35, 125, 14]. For example, Hu et al. [34] pre-trained a GNN at both the node and graph levels so that local and global information could be learned together. Other work combines contrastive learning across multiple granularities [38, 113].

The advantage of multi-task pre-training is breadth: the encoder is less tied to a single pretext task. The drawback is optimization complexity. Different objectives may compete with each other, and the resulting model may underperform on some tasks even if aggregate pre-training loss improves. For this reason, multi-task pre-training is best understood as a broader design space rather than a guaranteed remedy for negative transfer. This line of thinking now overlaps with multi-graph and multi-domain pre-training, where the goal is to build encoders that transfer across graph collections rather than within a single benchmark only [44, 122].

4.5 Further Discussion

Pre-training is therefore a necessary backdrop for graph prompting, but it does not by itself solve the transfer problem. The main value of prompting is that it offers a lighter adaptation interface on top of a pre-trained encoder. Instead of updating the full model for every downstream task, prompting modifies the input graph, the task interface,

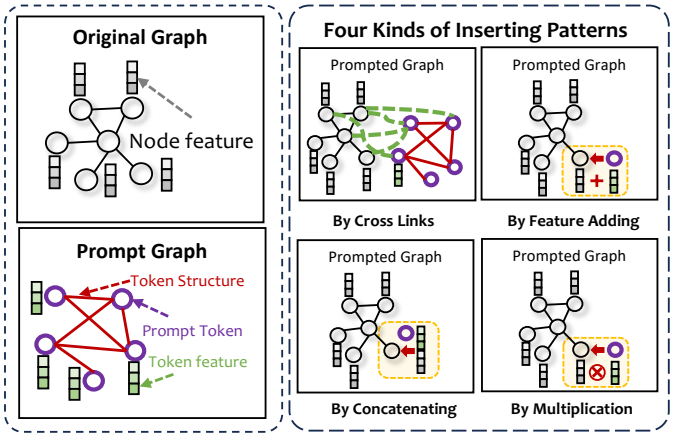


Fig. 7: Prompt Tokens, Structures, and Inserting Patterns.

or a small set of prompt parameters so that the downstream problem can better match what the encoder already knows.

This does not mean that prompting uniformly outperforms fine-tuning. Its effectiveness still depends on the scope and quality of pre-training, the gap between the pretext task and the downstream task, and the amount of supervision available during adaptation. In some settings, prompting can provide a more parameter-efficient route to transfer; in others, full or partial fine-tuning remains stronger. For this reason, graph prompting should be viewed as a complement to pre-training and fine-tuning, not as a universal replacement. Recent benchmark studies reinforce this point by showing that broader and more standardized evaluation is still needed for graph foundation models and their downstream adaptation behavior [115, 123].

5 Prompt Design for Graph Tasks

In this section, we propose a unified view for the graph prompt. As shown in Figure 7, the graph prompt should contain at least three components: prompt tokens with prompt vector; token structures preserving inner correlations of these tokens; and inserting patterns indicating how to integrate the original graph with prompts. Beyond these details, we are particularly interested in the following questions: *Question 1:* How do these works design the graph prompt? *Question 2:* How do these works reformulate downstream tasks to the pre-training tasks? *Question 3:* How do these works learn an effective prompt? and *Question 4:* What are the inner connections of these works, their advantages and limitations? With these questions, we summarize the most representative works published recently and present them in Table 2.

5.1 Prompt Token, Structure, and Inserting Pattern

Question 1: How do They Design A Graph Prompt?

A. Prompt as Tokens. The simplest graph prompt can be treated as some additional features added to the original graph features [137]. For example, given a graph feature matrix $\mathbf{X} = \{x_1, \cdots, x_N\} \in \mathbb{R}^{N\times d}$ where $x_i \in \mathbb{R}^{1\times d}$ is the feature of i-th node and d is the dimension of the feature space. Fang et al. [12] and Shirkavand and Huang [79] treat the basic prompt as a learnable vector $p \in \mathbb{R}^{1\times d}$, which can be added to all node features and make the manipulated feature matrix be $\mathbf{X}^* = \{x_1 + p, \cdots, x_N + p\}$. In this way, we can use the reformulated features to replace the original features and process the graph with pre-trained graph models. A later work [13] further extends one prompt token to multiple tokens and makes the performance better. PGCL [21] design a prompt vector for semantic view and another prompt vector for contextual view, then they add these prompt vectors to the graph-level representations by element-wise multiplication. A similar prompt design is also adopted in VNT [89]. The difference is that their inserting pattern does not add the prompt token to the original graph feature but concatenates the prompt token with the original node set and tries to integrate them by the self-attention function in a graph transformer. In GraphPrompt [56], the prompt token is similar to the format defined by Fang et al. [12]. The difference thing is that previous work designed the prompt tokens in the initial feature space, while this method assumes the prompt in the hidden layer of the graph model. Usually, the hidden size will be smaller than the original feature, making their prompt token shorter than the previous one. Another different thing is that the graph prompt here is used to assist graph pooling operation (a.k.a Readout). For example, given the node set $\mathcal{V} = \{v_1, \cdots, v_{|\mathcal{V}|}\}$, the embedding of node v is $\mathbf{h}_v \in \mathbb{R}^{1\times d}$, a prompt token $\mathbf{p}_t \in \mathbb{R}^{1\times d}$ specified to task t is inserted to the graph nodes by the element-wise multiplication ($\otimes$): $\mathbf{s}_t = \text{Readout}(\{\mathbf{p}_t \otimes \mathbf{h}_v : v \in \mathcal{V}\})$. Similarly, SGL-PT [136] creates a prompt token to connect to all nodes in the graph. The prompt token preserves a global perception of the graph and can assist their global branch of the pre-training tasks, which can be also treated as a pooling strategy. Aiming at aligning node classification and link prediction, GPPT [84] defines graph prompts as additional tokens that contain task tokens and structure tokens. Here the task token refers to the description of the downstream node label to be classified and the structure token is the representation of the subgraph surrounding the target node. By this means, predicting node v's label can be reformulated to predict a potential link between node v's structure token and the label task token. Aiming at the node classification task, ULTRA-DP [4] creates a prompt token for each target node, where the token feature is the weighted sum of position embedding of the target node and a task embedding of the pre-training task. HetGPT [60] design a prompt with node tokens and class tokens, which are organized in a similar way to GPPT. The difference is that they also add a type-specific feature token to make graph prompts sensitive to different node types, by which they can extend existing graph prompts to heterogeneous graphs.

B. Prompt as Graphs. The graph prompt in All in One [86] is an additional subgraph that can be learned by efficient tuning. The prompt tokens are some additional nodes that have the same size of node representation as the original nodes. They assume the prompt tokens should be in the same semantic space as the original node features so that we can easily manipulate node features with these tokens. The token structures include two parts. The first part is the inner links among different tokens, and the second part is the cross-links between the prompt graph and the original graph. These links can be pre-calculated by the dot product between one token to another token (inner links) or one

TABLE 2: Summary of existing representative works on graph prompt. $\mathcal{S}$: Subgraph. $V(\mathcal{S})$: Node set within subgraph $\mathcal{S}$. π: Pre-trained parameters. ϕ: Task head parameters. θ: Prompt parameters. $\tilde{\mathbf{s}}$: Filled prompt.

	pre-training task			prompt design			downstream tasks			answering function	
Paper	node	edge	graph	prompt components	inserting pattern	prompt tuning	node	edge	graph	Preset	Learnable
GPPT (KDD 2022 [84])	✗	✓	✗	structure token: $\mathbf{s}_v \in \mathbb{R}^d$ task token: $\mathbf{c}_y \in \mathbb{R}^d$	$\mathbf{s}_{v_i} \leftarrow f_\theta(v_i)$ $\tilde{\mathbf{s}}_{y,v_i} \leftarrow [\mathbf{c}_y, \mathbf{s}_{v_i}]$	Cross Entropy	✓	✗	✗	✓	✗
GPF (arXiv [12])	✓	✓	✓	prompt feature $\mathbf{p} \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i \leftarrow \mathbf{x}_i + \mathbf{p}$	$\max_{\mathbf{p},\phi} \sum_{(y_i,\tilde{\mathbf{s}}_i)} p_{\pi,\phi}(y_i\|\tilde{\mathbf{s}}_i)$	✗	✗	✓	✗	✓
All in One (KDD 2023 [86])	✗	✗	✓	prompt token: $\mathcal{P} = \{\mathbf{p}_1, ..., \mathbf{p}_{\|\mathcal{P}\|}\}$ token structure: $\{(\mathbf{p}_i, \mathbf{p}_j)\|\mathbf{p}_i, \mathbf{p}_j \in \mathcal{P}\}$	$w_{ik} \leftarrow \sigma(\mathbf{p}_k \cdot \mathbf{x}_i^T)$ if $\sigma(\mathbf{p}_k \cdot \mathbf{x}_i^T) > \delta$ else 0 $\tilde{\mathbf{s}}_i \leftarrow \mathbf{x}_i + \sum_{k=1}^{\|\mathcal{P}\|} w_{ik}\mathbf{p}_k$	Meta-Learning	✓	✓	✓	✓	✓
GraphPrompt (WWW 2023[56])	✗	✓	✗	prompt token: $\mathbf{p}_t \in \mathbb{R}^d, t \in \mathcal{T}$ structure token: $\mathbf{s} \in \mathbb{R}^d$ task token: $\mathbf{c}_y \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i^t \leftarrow \text{Readout}(\{\mathbf{p}_t \odot f_\pi(v)\|$ $v \in V(\mathcal{S}_i)\})$ $\mathbf{c}_y \leftarrow \text{Mean}(\{\tilde{\mathbf{s}}_j^t \| y_j = y\})$	$\min_{\mathbf{p}_t} -\sum_{(y_i,\mathcal{S}_i)} \ln \frac{\exp(\text{sim}(\tilde{\mathbf{s}}_i^t, \mathbf{c}_{y_i})/\tau)}{\sum_{y\in Y}\exp(\text{sim}(\tilde{\mathbf{s}}_i^t, \mathbf{c}_y)/\tau)}$	✓	✗	✓	✓	✗
PGCL (arXiv [21])	✗	✗	✓	semantic token: $\mathbf{p}^s \in \mathbb{R}^d$ contextual token: $\mathbf{p}^c \in \mathbb{R}^d$	$z_x^{ps} = z_x^s \odot \mathbf{p}^s$ $z_x^{pc} = z_x^c \odot \mathbf{p}^c$	$\min -\sum_{(v,a,b)\in\mathcal{T}} \log \frac{\exp(\text{sim}(z_v^p, z_a^p)/\tau)}{\sum_{u\in\{x,y\}}\exp(\text{sim}(z_v^p, z_u^p)/\tau)}$	✓	✓	✓	✓	✗
PRODIGY (NeurIPS 2023 [36])	✗	✓	✗	data graph: $\mathcal{G}^D \sim \oplus_{i=1}^K \tilde{\mathcal{N}}(\mathcal{V}_i, \mathcal{G})_\rangle$ task graph: $\mathcal{G}^T$	$\tilde{\mathbf{s}}_i \leftarrow f_{\pi^T}^T(\mathcal{G}^T \| f_{\pi^D}^D(\mathcal{G}^D))$	Fixed	✓	✓	✓	✓	✗
SGL-PT (arXiv [136])	✓	✗	✓	prompt token: one vector for each graph	connect to all nodes in the graph	contrastive loss and reconstruction loss	✓	✗	✗	✓	✗
GPF-Plus (NeurIPS 2023 [13])	✓	✓	✓	prompt features $\mathbf{p}_1, \cdots, \mathbf{p}_k \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i \leftarrow \mathbf{x}_i + \sigma(\mathbf{p}_1, \cdots, \mathbf{p}_k)$	$\max_{\mathbf{p},\phi} \sum_{(y_i,\tilde{\mathbf{s}}_i)} p_{\pi,\phi}(y_i\|\tilde{\mathbf{s}}_i)$	✗	✗	✓	✗	✓
DeepGPT (arXiv [79])	✓	✓	✓	prompt token: $\mathbf{p} \in \mathbb{R}^d$ prefix token: $\mathbf{P} \in \mathbb{R}^{\|\mathcal{P}\|\times d}$	$\tilde{\mathbf{x}}_i \leftarrow \mathbf{x}_i + \mathbf{p}$ $\tilde{\mathbf{s}}_i \leftarrow f_{\mathbf{P},\pi}(G, \tilde{\mathbf{x}}_i)$	$\min_{\mathbf{p},\mathbf{P},\phi} \sum_{(y_i,v_i)} \mathcal{L}(p_\phi(\tilde{\mathbf{s}}_i), y_i)$	✗	✗	✓	✗	✓
ULTRA-DP (arXiv [4])	✗	✓	✗	prompt token: $\mathbf{p}_i = \mathbf{p}^{\text{task}} + w^{\text{pos}}\mathbf{p}_i^{\text{pos}}$, $\mathbf{p}_i^{\text{pos}}$ denotes v_i's positional embedding	create a virtual node v_i^p for target node v_i, $G' \leftarrow (\mathcal{V} \cup \{v_i^p\}, \mathcal{E} \cup \{(v_i^p, v_i)\}, \mathbf{X} \cup \{\mathbf{p}_i\})$	Multi-task-Learning	✓	✗	✗	✗	✓
HetGPT (arXiv [60])	✗	✓	✗	prompt token: $\mathcal{F} = \{\mathbf{f}_i^A\}_{i=1}^K$ for $A \in \mathcal{A}$ task token: $\mathbf{c}_y \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i^A \leftarrow \mathbf{x}_i^A + \sum_{k=1}^K w_{ik}\mathbf{f}_k^A$, $\mathbf{z}_i = f_\pi(G, \tilde{\mathbf{s}}_i^A)$	$\min_{\mathbf{C},\mathcal{F}} -\sum_{(y_i,v_i)} \log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{c}_{y_i})/\tau)}{\sum_{y\in\mathbf{Y}}\exp(\text{sim}(\mathbf{z}_i, \mathbf{c}_y)/\tau)}$	✓	✗	✗	✓	✗
SAP (arXiv [20])	✓	✗	✗	task token: $\mathcal{P} = \{\mathbf{c}_y\}_{y\in\mathbf{Y}}$ structure token: $\mathcal{W} = \{(v_i, c_j)\}_{v_i\in\mathcal{V}, c_j\in\mathcal{P}}$	$G' \leftarrow (\mathcal{V} \cup \mathcal{P}, \mathcal{E} \cup \mathcal{W})$ $\mathbf{Z}^{(1)} = \text{MLP}_{\pi'}(\mathbf{X})$ $\mathbf{Z}^{(2)} = \text{GNN}_{\pi''}([\mathbf{X}, \mathbf{P}], [\mathbf{A}, \mathbf{W}])$	$\min_{\mathbf{W}} -\sum_{(y_i,v_i)} \log \frac{\exp(\text{sim}(\mathbf{z}_i^{(1)}, \mathbf{z}_{y_i}^{(2)})/\tau)}{\sum_{y\in\mathbf{Y}}\exp(\text{sim}(\mathbf{z}_i^{(1)}, \mathbf{z}_y^{(2)})/\tau)}$	✓	✗	✓	✓	✗
VNT (KDD 2023 [89])	✓	✓	✗	$\boldsymbol{P} = [\boldsymbol{p}_1; \ldots; \boldsymbol{p}_P]$, $\boldsymbol{p}_p \in \mathbb{R}^F$	$\left[E^1 \| Z^1\right] = L^1\left(\left[E^0 \| P\right]\right) \in \mathbb{R}^{(V+P)\times F}$	Cross Entropy	✓	✗	✗	✗	✓

token to another original node (cross links). The inserting pattern is to add the prompt graph to the original graph by the cross-links and then treat the combined graph as a new graph, and send it to the pre-trained graph model to get the graph-level representation. The prompt graph in PRODIGY [36] includes data graphs and a task graph. The data graphs can be treated as subgraphs surrounding the target nodes (for node classification task), node pairs (for edge classification task), or just denoted as the graph classification instance. Here the prompt tokens and prompt structures are just the same as in the original graph. The task graph contains data tokens and label tokens where each data token connects to one data graph and is further connected by label tokens. Unlike previous works that aim at reformulating downstream tasks to the pre-training task by prompting the downstream data, PRODIGY leverages the prompt graph to unify all the upstream and downstream tasks. Their pre-training strategy is a set of tasks including neighboring matching and label matching, which can be reformulated as predicting the similarity between the data token and the label token in the prompt graph. SAP [20] also connects several prompt tokens (each token corresponds to one class) to the original graph by cross-links defined in All in One. The difference is that their prompting task is a node-level contrastive task, in which they use MLP to encode the node features as the first view and they use a GNN to encode the prompted graph as the second view, which is consistent with their pre-training task.

5.2 Aligning Tasks by Answering Function

Question 2: How do they reformulate downstream tasks to the pretext?

A. Handling Different Level Tasks. All in One [86] pre-trains the graph model via graph-level contrastive learning. The pre-training task aims to learn a robust graph encoder over different graph views generated by the perturbations of the original graph. To reformulate various graph tasks to this graph-level pretext, they first unify node-level, edge-level, and graph-level tasks to the graph-level task by induced subgraphs, which are also introduced in PRODIGY [36, 56, 21]. Then they claim that the graph prompt added to the original graph is in nature the simulation of any graph operations such as node feature masking, node or edge perturbations, subgraph removing, etc. With this opinion, we just need to add an appropriate graph prompt to downstream graph datasets then it will be promising to further reformulate the downstream task to the pretext.

B. Learnable Answering Function. To output the results of downstream tasks, Sun et al. [86] design two types of

answering functions. The first one is a learnable task head (such as an MLP mapping function) that can be easily tuned with very limited data. It takes the graph-level representation generated by the pre-trained graph model and then outputs the downstream result. For example, if the downstream is a three-class node classification, we can simply use a dense layer with three hidden units to connect the graph representation, which is generated by the pre-trained model on the combined graph with node included graph and a graph prompt. In this case, both the graph prompt and the task head are tunable, so we can adjust them alternately. Similar learnable answering functions are also adopted in other works like [12, 13, 89]. The good point is that they are very easy to align two tasks, however, it also increases the tuning workflow.

C. Hand-crafted Answering Function. To further reduce the tuning burden, All in One [86] also proposes a second answering function, which is hand-crafted without any trainable task head. For example, for a node classification task, we can set up K unique sub-prompts, each aligning with a different node type, where K represents the total number of node categories. If the pre-training involves a task like GraphCL [119], which aims to maximize similarity at the graph level between pairs of graph views, then the target node can be classified with label $\ell, (\ell = 1, 2, \cdots, K)$ if the ℓ-th graph most closely resembles the original node-inclusive graph. Similarly, GPPT [84] and HetGPT [60] use link prediction as their pre-training task and reformulate downstream node classification by unifying it as the same task template. For example, by treating the node label as an additional token, we can use the pre-trained model to directly output the possibility of an edge between the label token and the target node. The pre-training strategy in SAP [20] is to compare node representations from two graph views, the first of which is generated by node feature encoding and the second of which is encoded by a graph model. To this end, their prompt tokens denote class information and they compare node representation with each class token to find the class with the largest similarity as the inference results. By designing a unified task template, PRODIGY [36] uses a hand-crafted graph prompt to describe all node, edge, and graph classification tasks by predicting the link between data tokens and label tokens. Liu et al. [56] extend the link prediction task as graph pair similarity and treat it as their pre-training task, to align the downstream node classification and graph classification task, they design a unified answering template making the downstream side aligned with the pre-training side. Specifically, given a triplet of induced graphs (g_1, g_2, g_3) where g_1 and g_2 have the same label, g_1 and g_3 have different labels. In particular, when the target task is node classification, the induced graph refers to the contextual subgraph of the target node. The unified answering template is defined as $\text{sim}(g_1, g_2) > \text{sim}(g_1, g_3)$.

5.3 Prompt Tuning

Question 3: How do They Learn A Graph Prompt?

A. Meta-Learning Technique. To learn appropriate prompts, Sun et al. [86] leverage meta-learning techniques (such as MAML[16] model) to obtain a robust starting point for the prompt parameters. Since the support set and query set include various graph tasks (such as node classification, link prediction, graph classification, etc), the learned graph prompt is expected to be more general on various downstream tasks.

B. Task-specific Tuning. Besides All in One [86], which aims to learn a general prompt on various downstream tasks, there are also some works that target specific downstream tasks such as graph classification. In this case, the prompt tuning can be more task-directed. For example, GPF [12] aims at a graph classification task, so it just needs to tune the prompt token $\mathbf{p}$ and the task head ϕ by maximizing the likelihood of predicted correct graph labels given the prompted graph representation $\tilde{\mathbf{s}}_i$ from the pre-trained graph model π. In this case, the task head tuning and the prompt tuning share the same objectives, which can be formulated by: $\max_{\mathbf{p},\phi} \sum_{(y_i,\tilde{\mathbf{s}}_i)} p_{\pi,\phi}(y_i|\tilde{\mathbf{s}}_i)$.

C. Tuning in Line with Pretext. Intuitively, prompting aims at reformulating downstream tasks to the pre-training task. Therefore, it would be more natural if the prompt tuning shares the same objective with the pre-training task. As suggested in GraphPrompt [56], the authors use a similar loss function to learn prompts. Similarly, GPPT [84] and VNT [89] adopt the same loss function (Cross-Entropy) as their link prediction and node classification tasks, respectively. HetGPT [60] and SAP [20] use a node-level contrastive loss to learn their prompt tokens because their pre-training task is also conducted by the same contrastive task (node pair comparison). PGCL [21] introduces graph-level loss to align with the pre-training task. ULTRA-DP [4] develop two pre-training tasks including edge prediction and neighboring prediction, each of which corresponds to one task embedding. In the pre-training phase, they randomly select a task and then integrate specific task-related embeddings into the prompt tokens. These learnable task embeddings are then trained with the graph model.

5.4 Further Discussion

Question 4: What are Their Connections, Pros and Cons?

The good point of GPF [12] is that they propose a very simple prompt that can be easily used in various pre-training tasks and downstream tasks. However, a single prompt token added to all nodes is very limited in expressiveness and generalization. A potential solution is to learn an independent prompt token for each node, which means one node corresponds to one prompt token, but this will cause low efficiency in parameters. To this end, we can train K-independent basis vectors and use them to compound each node token (GPF-Plus [13]). This improvement makes their work have more similar insights with All in One [86].

HetGPT [60] extends prompt tokens to type-specific format, which can deal with graph prompting in heterogeneous data. However, they can only deal with node classification tasks. To this end, GraphPrompt [56] reformulates link prediction to graph pair similarity task. It is worth noticing that the role of their prompt token is very similar to the project vector in the graph attention network. There are also some attention-based graph-pooling methods, which share the same motivation. The difference is that the authors claim the graph-pooling component in the pre-training stage might not fit other downstream tasks, thus needing additional

prompts to redirect the graph-pooling component in the graph model.

GPPT [84] represents a specific instance within the broader framework of All in One [86]. For instance, if we minimize the prompt graph to isolated tokens that correlate with node classes and substitute the resulting graphs with a complete graph, the All in One prompt structure can be simplified into the GPPT format. This allows for the utilization of edge-level pretexts in node classification tasks within the GPPT framework. The shortcoming of GPPT might be that it is restricted to binary edge prediction pretexts and is solely effective for node classification in downstream tasks. In comparison, frameworks like GraphPrompt and All in One are designed to accommodate a wider array of graph-related tasks, extending beyond just node-level classification. The good point is that when adapting models for different tasks, GraphPrompt, GPF, and GPF-Plus often require the tuning of an extra task-specific module. In contrast, All in One, and GPPT utilize task templates that focus more on the manipulation of input data and are less dependent on the specifics of downstream tasks.

Intuitively, the data graphs, one of the components in PRODIGY [36], are very similar to the induced graph in All in One and GraphPrompt. The pre-training task in PRODIGY can be seen as predicting a link between the data token and the label token, which shares a similar insight with GPPT. The good thing is that their prompts have no trainable parameters, which means they do not need to tune the prompt and are more efficient in the in-context learning area. PRODIGY does not need any tuning work and can be used in knowledge transferring between different datasets. However, a non-tunable prompt is usually not flexible enough, which might also limit the generalization of the pre-trained model when the downstream tasks to be transferred are located in a different domain from the pre-training one. In contrast, ULTRA-DP [4] tune prompt both in the pre-training stage and the downstream tasks. It first put the prompt tuning work in the pre-training stage to obtain the task embeddings, which are one of the main components in their prompt. Then they use these task embeddings to initialize a downstream prompt. Intuitively, their prompts are not used to reformulate downstream tasks to the pretext. Instead, these prompt tokens are used to select suitable pre-training tasks from a task pool to fit the downstream task. It is still an interesting question of how to achieve the optimal balance given efficiency, generalization, and the flexibility of prompt.

Compared with other works that usually define clear inserting patterns, VNT [89] concatenates prompt tokens with the original node set and then puts all of them into the graph transformer. Actually, the graph transformer will leverage a self-attention function to further calculate the correlations among them, which can also be treated as a variant of inserting patterns defined in All in One. The good thing is that we do not need to design a threshold to tailor the connection but the shortcoming is that it can only use a graph transformer as its backbone and can not applied to more message-passing-based graph models. In addition, there are also some more advanced variants of graph transformers requiring additional position embedding as one of their input. However, the prompt tokens in VNT have no clear inserting links to the original graph, which might not make it easy to apply existing position encoding approaches for these graph transformer variants.

6 Graph Prompting in Multi-Modal and Multi-Domain Areas

6.1 Multi-Modal Prompting with Graphs

The fusion of images, sound, and text has been widely studied and achieved remarkable success. However, most of these modalities are described by linear data structure. In our real-world life, there are more kinds of data in non-linear structures like graphs. How to connect these linear modalities (e.g. text, images, sound, etc) to the non-linear modalities (e.g. graphs) has become a very attractive research topic because it is a bigger move towards artificial general intelligence. Unfortunately, reaching this vision is very tough. Currently, we only see some hard progress in the fusion of text and graphs, especially in the text-attributed graphs. With the help of recent large language models, the fusion of text and graph has achieved even more notable performance. Since there have already been some informative surveys on this topic, we next briefly discuss some representative works that are closely related to *prompt techniques*. We suggest readers refer to the mentioned literature [50, 52] to require more detailed information further.

A. Prompt in Text-Attributed Graphs. Wen and Fang [104] ventured into enhancing text classification in scenarios with limited data resources by the proposed model, Graph-Grounded Pre-training and Prompting (G2P2). Their work identifies the issue of insufficient labeled data in supervised learning and proposes a solution leveraging the inherent network structure of text data, such as hyperlinks or citation networks. G2P2 utilizes text-graph interaction strategies with contrastive learning during the pre-training phase, followed by an inventive prompting technique during the classification phase, demonstrating notable effectiveness in zero- and few-shot text classification tasks. Zhao et al. [132] focused on molecule property prediction, a field grappling with the scarcity of labeled data. Their study introduces an integrated graph-text model to enhance prompt-based molecule task learning in a zero-shot context. This model employs generalized position embedding and decouples encoding of the graph from task prompt, enhancing its generalization capability across novel tasks. Li and Hooi [49] explored node classification within the framework of multi-modal data (text and graph), particularly focusing on limited-label scenarios. Unlike traditional works that usually feed pre-computed text features into graph neural networks, they incorporate raw texts and graph topology by a hand-crafted language prompt template into the model design.

B. Large Language Models in Graph Data Processing. Chen et al. [6] explored the potential of Large Language Models (LLMs) in graph node classification tasks. They investigated two pipelines: LLMs-as-Enhancers, which enhances node text attributes using LLMs followed by predictions via Graph Neural Networks (GNNs), and LLMs-as-Predictors, which directly employs LLMs as standalone predictors. Their empirical evaluations revealed that deep sentence embedding models and text-level augmentation through LLMs effectively enhance node attributes, while LLMs also show

promise as standalone predictors, albeit with concerns about accuracy and test data leakage. Fatemi et al. [15] conducted a comprehensive study on encoding graph-structured data for consumption by LLMs. Their findings include the influence of the graph encoding method, the nature of the graph task, and the structure of the graph on LLM performance. They demonstrated that simple prompts are most effective for basic graph tasks and that graph encoding functions significantly impact LLM reasoning. Their experimental setup introduced modifications to the graph encoding function, revealing improvements in performance and demonstrating the effect of model capacity on graph reasoning ability. Jin et al. [40] introduced an innovative approach to pre-train language models on networks rich in text. Their framework, named PATTON, focuses on integrating the intricacies of textual attributes with the underlying network structure. They developed two novel pretraining strategies: one concentrating on the context within networks for masked language modeling, and the other on predicting masked nodes, thus capturing the interplay between text and network structure. The effectiveness of PATTON was validated through various experiments, showcasing its superior performance over traditional text/graph pretraining methods in diverse tasks such as document classification, retrieval, and link prediction in different domain datasets. This approach signifies a shift in pretraining methodologies, emphasizing the synergy between textual data and network context. Wang et al. [102] proposed a Knowledge Graph Prompting (KGP) method to enhance LLMs for multi-document question answering (MD-QA). They created a knowledge graph over multiple documents, with nodes representing passages or document structures and edges denoting semantic/lexical similarity. The LM-guided graph traverser in KGP navigates the graph to gather supporting passages, aiding LLMs in MD-QA. Their experiments indicated that the construction of KGs and the design of the LM-guided graph traverser significantly impact MD-QA performance.

C. Multi-modal Fusion with Graph and Prompting. The integration of multi-modal data using graph and prompting techniques has seen remarkable progress in recent years. For example, Edwards et al. [11] propose SynerGPT in the field of drug synergy prediction. This model leverages a transformer-based approach, uniquely combining in-context learning with genetic algorithms to predict drug synergies. In the area of vision-language models, Li et al. [48] develop GraphAdapter, a prompt-based strategy that utilizes an adapter-style tuning mechanism, bringing together textual and visual modalities through a dual knowledge graph. Liu et al. [54] extend work multi-modal fusion into molecular science with their proposed GIT-Mol. A large language model integrates graph, image, and textual data with the help of prompt, offering substantial improvements in various tasks like molecule generation and property prediction. Although much effort has been made in the past few years, the academic is still trying hard to find better solutions to integrate text and graphs via text-attributed graphs or knowledge graphs. There is still a very large imagination in the fusion of more kinds of modalities.

6.2 Graph Domain Adaptation with Prompting

The field of graph domain adaptation has seen significant advancements, particularly with the integration of prompting techniques. However, graph domain adaptation is still not a well-solved problem because there exist at least two challenges: The first one is how to align semantic spaces from different domains. The second one is how to identify structural differences.

A. Semantic Alignment. In particular, All in One [86] extends the "pre-training and fine-tuning" workflow with multi-task prompting for GNNs, unifying prompt formats, and introducing meta-learning for prompt optimization. To make the graph model adaptive to different graph domains, they first reveal that the graph prompt in nature can be seen as graph operation and then they use graph prompt to manipulate different domain graph datasets. GraphControl [137] introduces a unique deployment module inspired by ControlNet, effectively integrating downstream-specific information as conditional inputs to enhance the adaptability of pre-trained models to target data. This approach aligns input space across various graphs and incorporates unique characteristics of the target data. Zhang et al. [129] presents a pre-training model for knowledge graph transfer learning. This model uses a general prompt-tuning mechanism, treating task data as a triple prompt, enabling flexible interactions between task KGs and task data. Yi et al. [118] combines personalized graph prompts with contrastive learning for efficient and effective cross-domain recommendation, particularly in cold-start scenarios. A representative work is proposed by Liu et al. [51], in which they describe graph nodes from different domains by language and then use LLM to get a textual embedding. However, this work needs the semantic name of each feature while sometimes graph features are usually latent vectors without clear semantic meaning.

B. Structural Alignment. Cao et al. [3] analyze the feasibility between different graph datasets and found that the structural gap holds the upper bound of various graph model transferability. GraphGLOW [133] addresses the limitations of existing models that operate under a closed-world assumption, where the testing graph is identical to the training graph. This approach often leads to prohibitive computational costs and overfitting risks due to the need for training a structure learning model from scratch for each graph dataset. To this end, it coordinates a single graph-shared structure learner with multiple graph-specific GNNs to capture generalizable patterns of optimal message-passing topology across datasets. The structure learner, once trained, can produce adaptive structures for unseen target graphs without fine-tuning, thereby significantly reducing training time and computational resources. AAGOD by Guo et al. [23] proposes a data-centric framework for OOD detection in GNNs. They use a parameterized amplifier matrix, which is treated as a prompt, to superimpose on the adjacency matrix of input graphs. Inspired by prompt tuning, Shirkavand and Huang [79] propose DeepGPT for graph transformer models. They add prompt tokens to the input graph and each transformer layer. By updating the prompt tokens, they can efficiently tune a graph model on different graph datasets. A recent work [131] leverages the graph prompt

capability in manipulating graph data and tries to integrate multiple graph datasets in the graph pre-training stage, the results from which strongly suggest the huge potential of transferring preserved knowledge from one domain to another.

7 Potential Applications

With the widespread utilization of networks as a data modeling structure for representing diverse relational information across social, natural, and academic domains, the graph prompt mechanism exhibits substantial potential for a wide range of real-world applications. In this section, we explore the potential applications of graph prompting in online social networks, recommender systems, knowledge management, and biology.

Online Social Networks. Online social platforms consist of users who can be represented as nodes, and their social connections form online social networks (OSNs). Previous research has investigated the potential of prompting mechanisms in identifying fake news within OSNs to prevent malicious attacks [108]. Specifically, they employ textual prompts applied to pre-trained language models (PLMs) to distill general semantic information. By combining this semantic signal with the dynamics of information propagation within social networks, improved classification performance can be achieved. While the use of tailored textual prompts for PLMs has been studied, the application of graph prompting mechanisms within social networks is still under-explored. In the future, it is promising to directly apply prompt tuning techniques to social networks, utilizing few-shot labels for tasks such as fake news detection or anomaly detection [106, 23], where the labeling process is laborious and requires domain expertise. By incorporating prompts directly within social networks, this approach can address the scarcity of labeled data and enhance the security and trustworthiness of online social networks. Some recent work [5] indicates that the graph prompt presents a significant performance in the analysis of temporal interactions, which is promising when applied to online social networks.

Recommender Systems. E-commerce platforms provide a valuable opportunity to leverage recommender systems for enhancing online services. While prompt tuning in recommender systems has received limited research attention, it holds significant potential [118, 117, 110, 25]. In [118], the graph prompt tuning technique is applied to cross-domain recommendation scenarios to address the challenges of domain adaptation. Specifically, when applying a pre-trained recommendation model to the target domain, extra prompt nodes are introduced to achieve both efficient and effective domain recommendation. Meanwhile, Yang et al. [117] propose personalized user prompts to bridge the gap between contrastive pretext [107, 120] to downstream recommendation task. They design different kinds of personalized prompts, in combination with pre-trained user embeddings to facilitate dynamic user representations, leading to more accurate and personalized recommending results. In the future, further exploration into the integration of graph prompt tuning within recommender systems can be conducted to enhance recommendation performance, personalization, and adaptability across different domains.

Knowledge Management. There are two branches of research focused on performing prompting on knowledge graph (KG) for improved knowledge management. The first branch involves direct prompting on knowledge graphs using a pre-trained KG model to facilitate knowledge transfer, enabling better generalization across different KG data and tasks [129]. The second branch explores the combination of grounded knowledge from KGs with the cognitive abilities of LLMs [102, 91, 72, 65, 128, 64] to improve performance in downstream tasks. In the first research line, Zhang et al. [129] proposed a structure pre-training and prompt tuning approach to realize knowledge transfer. They designed specific pre-training objectives to obtain a powerful KG model. Subsequently, a general prompt tuning technique was employed to facilitate knowledge transfer between task-specific KGs and data. In the second research line, prompt tuning techniques are adopted to combine the grounded knowledge inherent in KGs with LLMs for enhancing downstream tasks. For example, in [91], a novel graph neural prompting method was introduced to adapt KGs for LLMs by distilling valuable knowledge from KGs in a time- and parameter-efficient manner. Future research can further explore this trend by implementing more efficient graph prompting methods to fully distill beneficial knowledge from KGs to assist LLMs on diverse downstream tasks.

Biology. Molecules can be represented naturally as graphs, where atoms serve as nodes and chemical bonds act as edges [73]. Such graph modeling provides a basis for applying graph representation learning methods to perform tasks such as molecular property prediction, thereby benefiting scientific research and discovery [73, 57, 132, 11]. Previous research on graph representation learning in the molecular domain followed a task-specific approach. It involved training individual models tailored to specific molecule datasets [73, 82], lacking the generalization ability within the domain. Though there exist works that explored the utilization of LLMs (or LMs) as a universal tool for understanding molecules [69, 51], these approaches primarily rely on regular texts to describe molecules, overlooking the inner graph structures within molecules [130]. Meanwhile, some recent works have focused on investigating the co-modeling of graphs and languages to preserve both structural dependencies and achieve generalization abilities across tasks and datasets, even under few-shot or zero-shot settings [54, 132, 11]. For instance, in [54], the authors employed LoRA [33] to efficiently adapt to downstream tasks. Following recent work on graph prompt[19], which has achieved remarkable results in protein multimer structure prediction, we believe that graph prompting techniques can also be adopted to achieve more efficient task adaptation within the molecular domain.

8 ProG: A Unified Library for Graph Prompting

An indispensable component for fortifying the graph prompting ecosystem is a well-crafted tool. Despite the plethora of tools proposed for generalized graph learning, a notable absence persists in the realm of libraries dedicated to graph prompt functionalities. Addressing this gap, we are introducing `ProG` (Prompt Graph), an open-source, unified library

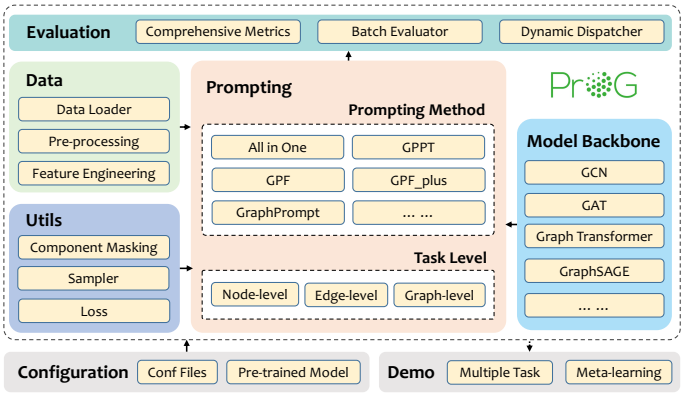


Fig. 8: The architecture of ProG.

meticulously designed to cater to the specific needs of graph prompting. This initiative promises to significantly enhance the landscape of graph-based applications by providing a versatile and comprehensive resource for researchers and practitioners alike.

ProG is a PyTorch-based library designed to facilitate single or multi-task prompting for pre-trained GNNs. The architecture is illustrated in Figure 8. It seamlessly integrates several widely used datasets in the graph prompt evaluation, including Cora, CiteSeer, Reddit, Amazon, and Pubmed etc. The tool is equipped with essential evaluation metrics such as Accuracy, F1 Score, and AUC score, commonly employed in various graph prompt-related tasks. Notably, ProG incorporates state-of-the-art methods like All in One [86], GraphPrompt [56], GPPT [84], GPF [12], and GPF-Plus [13], and it continues integrating more graph prompt models. In summary, ProG offers the following key features:

- **Quick Initiation.** All models are implemented within a consistent environment, accompanied by detailed demos, ensuring a swift initiation for newcomers.
- **Fully Modular.** ProG adopts a modular structure, empowering users to customize and construct models as needed.
- **Easy Extendable.** ProG is designed for seamless extension to encompass additional methods and a broader spectrum of downstream tasks, adapting to evolving research needs.
- **Standardized Evaluation.** ProG establishes a uniform set of evaluation processes, promoting equitable performance comparisons across models.

For additional information and access to the library, please visit the website of our library[3]. Additionally, we have curated a GitHub repository[4], serving as a centralized resource for the latest advancements in graph prompt learning. This repository includes a list of research papers, benchmark datasets, and available codes, fostering an environment conducive to ongoing research in this dynamic field. Regular real-time updates ensure that the repository remains current with emerging papers and associated codes.

3. https://github.com/sheldonresearch/ProG
4. https://github.com/WxxShirley/Awesome-Graph-Prompt

9 Challenges and Future Directions

9.1 Current Challenges

Graph prompt learning has made significant research progress, but it still encounters several challenges. In this subsection, we will discuss current challenges in detail.

Inherent Limitation within Graph Models Prompts, originated from the NLP field [2, 45, 53], serve as a means to unlock the potential of pre-trained language models for adapting to downstream tasks. This in-context learning ability emerges when the scale of model parameters reaches a certain level. For instance, widely known LLMs typically possess billions of parameters. With such a powerful pre-trained model, a simple textual prompt can distill specific knowledge for downstream tasks. However, when it comes to prompting on graph tasks, the conditions become more intractable since pre-trained graph models have significantly fewer parameters, making it challenging to harness their full downstream adaptation potential.

Intuitive Evaluation of Graph Prompt Learning In the NLP field, prompts are typically in a discrete textual format [2, 72], allowing for intuitive understanding, comparison, and explanation. However, existing graph prompts are represented as learnable tokens [84, 13, 12, 89, 60] or augmented graphs [86, 36, 20]. Such format poses challenges in intuitively understanding and interpreting graph prompts, as they lack a readable design. As a result, the effectiveness of prompts can only be evaluated based on downstream tasks, limiting efficient and comprehensive performance comparison among different kinds of graph prompts. Therefore, the development of a more intuitive graph prompt design with a readable format remains an open problem.

More Downstream Applications Currently, graph prompt learning is primarily applied to node or graph classification tasks on open-source benchmarks [24, 114]. Although potential applications have been discussed in Section 7, the real-world utilization of graph prompt learning remains limited. A notable example is its use in fraud detection within real-world transaction networks, addressing the issue of label scarcity [106]. However, compared to the widespread application of prompting techniques in the NLP domain [1, 72, 128], the potential of graph prompt learning in diverse real-world applications requires further exploration. Overcoming the main challenges of obtaining powerful domain-specific pre-trained graph models and designing suitable prompts for specific application scenarios that exhibit unique characteristics remains crucial.

Transferable Prompt Designs Existing studies on graph prompt learning [56, 84, 21, 13, 12, 20] typically focus on pre-training and prompt tuning using the same dataset, which limits the exploration of more transferable designs and empirical evaluation. Although PRODIGY [36] explores transferability within the same domain and All in One [86] provides empirical results regarding transferability across tasks and domains, the investigation of prompt learning across diverse domains and tasks remains limited. Achieving transferability across diverse tasks and domains requires aligning the task space [86], semantic features [137], and structural patterns [133], which necessitates further theoretical work to provide insights guiding the development of transferable prompt designs.

9.2 Future Directions

With the above analysis on graph prompt, we summarize future directions as follows:

Learning Knowledge from Large Graph Models (LGMs) like LLMs. Currently, graph models are typically tailored to specific domains or tasks, limiting generalization abilities across broader domains or tasks [52, 137]. Therefore, we are expecting the realization of large graph models (LGMs) as a universal tool to be intelligent enough to handle graph tasks across domains [51, 15]. A significant step towards this direction has been taken by Liu et al. [51], who proposed a powerful model (OFA) capable of addressing classification tasks for graphs coming from various domains. However, their approach still relies on LLMs as a general template to distill specific domain knowledge, which we believe can be replaced with suitable graph prompting techniques. Just as textual prompt has been widely used to adapt LLM for diverse applications [72, 102, 2], graph prompting techniques are promising to distill LGMs' knowledge specific for concrete downstream tasks. Applying the graph prompts to LGMs has the potential to revolutionize the field of deep graph learning, leveraging LGMs as a universal tool to tackle different tasks on graphs from diverse domains.

Transferable Learning. As we have discussed in Section 9.1, current graph prompt learning methods [86, 84, 36, 56, 89, 20, 79] are primarily limited to intra-dataset/domain settings, lacking the ability to transfer knowledge across different tasks or domains. While there have been efforts in graph transfer learning [137] to realize domain adaptation, research specifically focused on transferable graph prompting techniques remains limited. To enable the transfer ability, the prompts should be designed to realize the alignment between different task spaces [86], reformulating different tasks on graphs into a uniform template. Besides, both structural [133] and semantics alignment [137] should be realized via suitable graph prompts to enable domain adaptation. It is promising to implement transferable graph prompts, realizing knowledge transfer across domains to extend the generalization and applicability of graph models.

More Theoretical Foundation. Despite the great success of graph prompt learning on various downstream tasks, they mostly draw on the successful experience of prompt tuning on NLP domain [55, 2, 18, 92, 70, 78]. In other words, most existing graph prompt tuning methods are designed with intuition, and their performance gains are evaluated by empirical experiments. The lack of sufficient theoretical foundations behind the design has led to both performance bottlenecks and poor explainability. Therefore, we believe that building a solid theoretical foundation for graph prompt learning from a graph theory perspective and minimizing the gap between the theoretical foundation and empirical design is also a promising future direction.

More Explainable and Understandable Design. While existing graph prompt learning methods have demonstrated impressive results on various downstream tasks [23, 106, 117], we still lack a clear understanding of what exactly is being learned from the prompts. The black-box learning mode of prompt vectors raises questions about their interpretability and whether we can establish meaningful correspondences between the input data and the prompted graph [86]. These issues are crucial for understanding and interpreting prompts but are currently missing in most graph prompt research. To work towards trustworthy graph prompt learning, it is promising to explore the self-interpretability [8] to enable intuitive explanations of graph prompts. By gaining insights into the learned prompt vectors and structures, we can enhance our understanding of the underlying mechanisms and improve the interpretability of graph prompts. This, in turn, can lead to more effective utilization of prompts for security- or privacy-related downstream tasks.

10 Conclusion

In this survey, we explore the promising intersection between Artificial General Intelligence and graph data by graph prompt. Our unified framework has unveiled a structured understanding of graph prompts, dissecting them into tokens, token structures, and inserting patterns. This framework is a novel contribution, providing clarity and comprehensiveness for researchers and practitioners. By exploring the interplay between graph prompts and models, we've revealed fresh insights into the essence of graph prompts, highlighting their pivotal role in reshaping AI for graph data. With the development of ProG, a Python library, and a dedicated website, we've expanded the graph prompting ecosystem, enhancing collaboration and access to research, benchmark datasets, and code implementations. Our survey outlines a roadmap for the future. The challenges and future directions we've discussed serve as a beacon for the evolving field of graph prompting. With the above work, we hope our survey can push forward a new era of insights and applications in the AGI family.

Acknowledgments

This research is supported in part by project #MMT-p2-23 of the Shun Hing Institute of Advanced Engineering, The Chinese University of Hong Kong, and by grants from the Research Grant Council of the Hong Kong Special Administrative Region, China (No. CUHK 14217622), NSFC Grant No. U1936205. The first author, Dr. Xiangguo Sun, in particular, wants to thank his parents for their kind support during his tough period.

References

[1] A. M. Bran and P. Schwaller, "Transformers and Large Language Models for Chemistry and Drug Discovery," in *Drug Development Supported by Informatics*, 2024.

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal *et al.*, "Language Models Are Few-Shot Learners," in *NeurIPS*, vol. 33, 2020, pp. 1877–1901.

[3] Y. Cao, J. Xu, C. Yang, J. Wang, Y. Zhang, C. Wang, L. Chen, and Y. Yang, "When to pre-train graph neural networks? An answer from data generation perspective!" in *KDD*, 2023.

[4] M. Chen, Z. Liu, C. Liu, J. Li, Q. Mao, and J. Sun, "ULTRA-DP: Unifying Graph Pre-training with Multi-task Graph Dual Prompt," *arXiv preprint*, 2023.

[5] X. Chen, S. Zhang, Y. Xiong, X. Wu, J. Zhang, X. Sun, Y. Zhang, Y. Zhao, and Y. Kang, "Prompt learning on temporal interaction graphs," *arXiv preprint*, 2024.

[6] Z. Chen, H. Mao, H. Li, W. Jin, H. Wen, X. Wei, S. Wang, D. Yin, W. Fan, H. Liu, and J. Tang, "Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs," *ACM SIGKDD Explorations Newsletter*, 2024.

[7] J. Cheng, M. Li, J. Li, and F. Tsung, "Wiener graph deconvolutional network improves graph self-supervised learning," in *AAAI*, 2023, pp. 7131–7139.

[8] E. Dai and S. Wang, "Towards Self-Explainable Graph Neural Network," in *CIKM*, 2021, pp. 302–311.

[9] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering," in *NeurIPS*, vol. 29, 2016.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *arXiv preprint*, 2019.

[11] C. Edwards, A. Naik, T. Khot, M. Burke, H. Ji, and T. Hope, "SynerGPT: In-Context Learning for Personalized Drug Synergy Prediction and Drug Design," *arXiv preprint*, 2023.

[12] T. Fang, Y. Zhang, Y. Yang, and C. Wang, "Prompt tuning for graph neural networks," *arXiv preprint*, 2022.

[13] T. Fang, Y. Zhang, Y. Yang, C. Wang, and L. Chen, "Universal Prompt Tuning for Graph Neural Networks," in *NeurIPS*, 2023.

[14] X. Fang, L. Liu, J. Lei, D. He, S. Zhang, J. Zhou, F. Wang, H. Wu, and H. Wang, "Geometry-enhanced molecular representation learning for property prediction," *Nature Machine Intelligence*, vol. 4, no. 2, pp. 127–134, 2022.

[15] B. Fatemi, J. Halcrow, and B. Perozzi, "Talk like a graph: Encoding graphs for large language models," *arXiv preprint*, 2023.

[16] C. Finn, P. Abbeel, and S. Levine, "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks," in *ICML*, 2017.

[17] X. Fu, Z. Wang, Z. Chen, J. Li, Y. Zhu, Z. Lei, C. Shen, Y. Ye, C. Zhang, and J. Li, "Graph Prompting for Graph Learning Models: Recent Advances and Future Directions," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2506.08326v1

[18] T. Gao, A. Fisch, and D. Chen, "Making Pre-Trained Language Models Better Few-Shot Learners," in *ACL*, 2021, pp. 3816–3830.

[19] Z. Gao, X. Sun, Z. Liu, Y. Li, H. Cheng, and J. Li, "Protein Multimer Structure Prediction via Prompt Learning," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=OHpvivXrQr

[20] Q. Ge, Z. Zhao, Y. Liu, A. Cheng, X. Li, S. Wang, and D. Yin, "Enhancing Graph Neural Networks with Structure-Based Prompt," *arXiv preprint*, 2023.

[21] C. Gong, X. Li, J. Yu, C. Yao, J. Tan, C. Yu, and D. Yin, "Prompt Tuning for Multi-View Graph Contrastive Learning," *arXiv preprint*, 2023.

[22] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *KDD*, 2016, pp. 855–864.

[23] Y. Guo, C. Yang, Y. Chen, J. Liu, C. Shi, and J. Du, "A Data-Centric Framework to Endow Graph Neural Networks with Out-of-Distribution Detection Ability," in *KDD*, 2023, pp. 638–648.

[24] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *NeurIPS*, vol. 30, 2017.

[25] B. Hao, C. Yang, L. Guo, J. Yu, and H. Yin, "Motif-Based Prompt Learning for Universal Cross-Domain Recommendation," in *WSDM*, 2024.

[26] A. Hasanzadeh, E. Hajiramezanali, K. Narayanan, N. Duffield, M. Zhou, and X. Qian, "Semi-Implicit Graph Variational Auto-Encoders," in *NeurIPS*, vol. 32, 2019.

[27] K. Hassani and A. H. K. Ahmadi, "Contrastive Multi-View Representation Learning on Graphs," in *ICML*, vol. 119, 2020, pp. 4116–4126.

[28] A. Haviv, J. Berant, and A. Globerson, "BERTese: Learning to Speak to BERT," in *EACL*, 2021, pp. 3618–3623.

[29] M. He, Z. Wei, z. Huang, and H. Xu, "BernNet: Learning Arbitrary Graph Spectral Filters via Bernstein Approximation," in *NeurIPS*, vol. 34, 2021, pp. 14 239–14 251.

[30] Y. He, Y. Sui, X. He, and B. Hooi, "Unigraph: Learning a unified cross-domain foundation model for text-attributed graphs," *arXiv preprint*, 2024. [Online]. Available: https://arxiv.org/abs/2402.13630v3

[31] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, and J. Tang, "GraphMAE: Self-Supervised Masked Graph Autoencoders," in *KDD*, 2022, pp. 594–604.

[32] Z. Hou, Y. He, Y. Cen, X. Liu, Y. Dong, E. Kharlamov, and J. Tang, "GraphMAE2: A Decoding-Enhanced Masked Self-Supervised Graph Learner," in *The Web Conference*, 2023, pp. 737–746.

[33] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint*, 2021.

[34] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. S. Pande, and J. Leskovec, "Strategies for Pre-training Graph Neural Networks," in *ICLR*, 2020.

[35] Z. Hu, Y. Dong, K. Wang, K.-W. Chang, and Y. Sun, "GPT-GNN: Generative Pre-Training of Graph Neural Networks," in *KDD*, 2020, pp. 1857–1867.

[36] Q. Huang, H. Ren, P. Chen, G. Kržmanc, D. Zeng, P. Liang, and J. Leskovec, "PRODIGY: Enabling In-context Learning Over Graphs," in *NeurIPS*, 2023.

[37] X. Jiang, T. Jia, Y. Fang, C. Shi, Z. Lin, and H. Wang, "Pre-training on Large-Scale Heterogeneous Graph," in *KDD*, 2021, pp. 756–766.

[38] X. Jiang, Y. Lu, Y. Fang, and C. Shi, "Contrastive Pre-Training of GNNs on Heterogeneous Graphs," in *CIKM*, 2021, pp. 803–812.

[39] Z. Jiang, F. F. Xu, J. Araki, and G. Neubig, "How Can We Know What Language Models Know?" *TACL*, vol. 8, pp. 423–438, 2020.

[40] B. Jin, W. Zhang, Y. Zhang, Y. Meng, X. Zhang, Q. Zhu, and J. Han, "Patton: Language Model Pretraining on Text-Rich Networks," in *ACL*, 2023.

[41] M. Jin, Y. Zheng, Y.-F. Li, C. Gong, C. Zhou, and S. Pan, "Multi-Scale Contrastive Siamese Networks for Self-Supervised Graph Representation Learning," in *IJCAI*, 2021, pp. 1477–1483.

[42] W. Jin, T. Derr, Y. Wang, Y. Ma, Z. Liu, and J. Tang, "Node Similarity Preserving Graph Convolutional Net-

works," in *WSDM*, 2021, pp. 148–156.

[43] D. Kim and A. Oh, "How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision," in *ICLR*, 2021.

[44] D. Lachi, M. Azabou, V. Arora, and E. Dyer, "Graphfm: A generalist graph transformer that learns transferable representations across diverse domains," *arXiv preprint*, 2024. [Online]. Available: https://arxiv.org/abs/2407.11907v2

[45] B. Lester, R. Al-Rfou, and N. Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning," in *EMNLP*, 2021, pp. 3045–3059.

[46] J. Li, R. Wu, W. Sun, L. Chen, S. Tian, L. Zhu, C. Meng, Z. Zheng, and W. Wang, "What's Behind the Mask: Understanding Masked Graph Modeling for Graph Autoencoders," in *KDD*, 2023, pp. 1268–1279.

[47] X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," in *ACL-IJCNLP*, 2021, pp. 4582–4597.

[48] X. Li, D. Lian, Z. Lu, J. Bai, Z. Chen, and X. Wang, "GraphAdapter: Tuning Vision-Language Models With Dual Knowledge Graph," in *NeurIPS*, 2023.

[49] Y. Li and B. Hooi, "Prompt-Based Zero-and Few-Shot Node Classification: A Multimodal Approach," *arXiv preprint*, 2023.

[50] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, and J. X. Yu, "A survey of graph meets large language model: Progress and future directions," *arXiv preprint*, 2023.

[51] H. Liu, J. Feng, L. Kong, N. Liang, D. Tao, Y. Chen, and M. Zhang, "One for All: Towards Training One Graph Model for All Classification Tasks," *arXiv preprint*, 2023.

[52] J. Liu, C. Yang, Z. Lu, J. Chen, Y. Li, M. Zhang, T. Bai, Y. Fang, L. Sun, P. S. Yu, and C. Shi, "Towards Graph Foundation Models: A Survey and Beyond," *arXiv preprint*, 2023.

[53] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 195:1–195:35, 2023.

[54] P. Liu, Y. Ren, and Z. Ren, "GIT-Mol: A Multi-modal Large Language Model for Molecular Science with Graph, Image, and Text," *Computers in Biology and Medicine*, 2024.

[55] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, and J. Tang, "P-Tuning: Prompt Tuning Can Be Comparable to Fine-Tuning Across Scales and Tasks," in *ACL*, vol. 2, 2022, pp. 61–68.

[56] Z. Liu, X. Yu, Y. Fang, and X. Zhang, "Graphprompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks," in *The Web Conference*, 2023, pp. 417–428.

[57] Z. Liu, S. Li, Y. Luo, H. Fei, Y. Cao, K. Kawaguchi, X. Wang, and T.-S. Chua, "MolCA: Molecular Graph-Language Modeling with Cross-Modal Projector and Uni-Modal Adapter," in *EMNLP*, 2023.

[58] Q. Long, Y. Yan, P. Zhang, C. Fang, W. Cui, Z. Ning, M. Xiao, N. Cao, X. Luo, L. Xu, S. Jiang, Z. Fang, C. Chen, X.-S. Hua, and Y. Zhou, "Towards Graph Prompt Learning: A Survey and Beyond," *arXiv preprint*, 2024. [Online]. Available: https://arxiv.org/abs/2408.14520v3

[59] Y. Long, M. Wu, Y. Liu, Y. Fang, C. K. Kwoh, J. Chen, J. Luo, and X. Li, "Pre-training graph neural networks for link prediction in biomedical networks," *Bioinformatics*, vol. 38, no. 8, pp. 2254–2262, 2022.

[60] Y. Ma, N. Yan, J. Li, M. Mortazavi, and N. V. Chawla, "HetGPT: Harnessing the Power of Prompt Tuning in Pre-Trained Heterogeneous Graph Neural Networks," in *Proceedings of the ACM Web Conference 2024*, 2024.

[61] M. Niepert, M. Ahmed, and K. Kutzkov, "Learning Convolutional Neural Networks for Graphs," in *ICML*, 2016, pp. 2014–2023.

[62] M. Ou, P. Cui, J. Pei, Z. Zhang, and W. Zhu, "Asymmetric Transitivity Preserving Graph Embedding," in *KDD*, 2016, pp. 1105–1114.

[63] S. Pan, R. Hu, G. Long, J. Jiang, L. Yao, and C. Zhang, "Adversarially Regularized Graph Autoencoder for Graph Embedding," in *IJCAI*, 2018, pp. 2609–2615.

[64] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[65] J. Park, A. Patel, O. Z. Khan, H. J. Kim, and J.-K. Kim, "Graph-Guided Reasoning for Multi-Hop Question Answering in Large Language Models," *arXiv preprint*, 2023.

[66] J. Park, M. Lee, H. J. Chang, K. Lee, and J. Y. Choi, "Symmetric Graph Convolutional Autoencoder for Unsupervised Graph Representation Learning," in *ICCV*, 2019, pp. 6519–6528.

[67] Z. Peng, W. Huang, M. Luo, Q. Zheng, Y. Rong, T. Xu, and J. Huang, "Graph Representation Learning via Graphical Mutual Information Maximization," in *The Web Conference*, 2020, pp. 259–270.

[68] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *KDD*, 2014, pp. 701–710.

[69] C. Qian, H. Tang, Z. Yang, H. Liang, and Y. Liu, "Can Large Language Models Empower Molecular Property Prediction?" *arXiv preprint*, 2023.

[70] G. Qin and J. Eisner, "Learning How to Ask: Querying LMs with Mixtures of Soft Prompts," in *NAACL-HLT*, 2021, pp. 5203–5212.

[71] J. Qiu, Q. Chen, Y. Dong, J. Zhang, H. Yang, M. Ding, K. Wang, and J. Tang, "GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training," in *KDD*, 2020, pp. 1150–1160.

[72] J. Robinson, C. M. Rytting, and D. Wingate, "Leveraging Large Language Models for Multiple Choice Question Answering," *arXiv preprint*, 2023.

[73] Y. Rong, Y. Bian, T. Xu, W. Xie, Y. WEI, W. Huang, and J. Huang, "Self-Supervised Graph Transformer on Large-Scale Molecular Data," in *NeurIPS*, vol. 33, 2020, pp. 12 559–12 571.

[74] M. T. Rosenstein, Z. Marx, L. P. Kaelbling, and T. G. Dietterich, "To transfer or not to transfer," in *NeurIPS*, vol. 898, 2005.

[75] T. Schick and H. Schütze, "Few-Shot Text Generation with Natural Language Instructions," in *EMNLP*, 2021, pp. 390–402.

[76] ——, "It's Not Just Size That Matters: Small Language

Models Are Also Few-Shot Learners," in *NAACL-HLT*, 2021, pp. 2339–2352.

[77] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification," in *IJCAI*, vol. 2, 2021, pp. 1548–1554.

[78] T. Shin, Y. Razeghi, R. L. L. IV, E. Wallace, and S. Singh, "Autoprompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts," in *EMNLP*, 2020, pp. 4222–4235.

[79] R. Shirkavand and H. Huang, "Deep Prompt Tuning for Graph Transformers," *arXiv preprint*, 2023.

[80] Y. Song, H. Mao, J. Xiao, J. Liu, Z. Chen, W. Jin, C. Yang, J. Tang, and H. Liu, "A pure transformer pretraining framework on text-attributed graphs," *arXiv preprint*, 2024. [Online]. Available: https://arxiv.org/abs/2406.13873v1

[81] A. Subramonian, "MOTIF-Driven Contrastive Learning of Graph Representations," in *AAAI*, vol. 35, 2021, pp. 15 980–15 981.

[82] F.-Y. Sun, J. Hoffmann, V. Verma, and J. Tang, "InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via Mutual Information Maximization," in *ICLR*, 2020.

[83] M. Sun, J. Xing, H. Wang, B. Chen, and J. Zhou, "MoCL: Data-driven Molecular Fingerprint via Knowledge-aware Contrastive Learning from Molecular Graph," in *KDD*, 2021, pp. 3585–3594.

[84] M. Sun, K. Zhou, X. He, Y. Wang, and X. Wang, "GPPT: Graph Pre-Training and Prompt Tuning to Generalize Graph Neural Networks," in *KDD*, 2022, pp. 1717–1727.

[85] X. Sun, H. Yin, B. Liu, H. Chen, J. Cao, Y. Shao, and N. Q. Viet Hung, "Heterogeneous Hypergraph Embedding for Graph Classification," in *WSDM*, 2021, pp. 725–733.

[86] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan, "All in One: Multi-Task Prompting for Graph Neural Networks," in *KDD*, 2023, pp. 2120–2131.

[87] S. Suresh, P. Li, C. Hao, and J. Neville, "Adversarial Graph Augmentation to Improve Graph Contrastive Learning," in *NeurIPS*, vol. 34, 2021, pp. 15 920–15 933.

[88] Q. Tan, N. Liu, X. Huang, S.-H. Choi, L. Li, R. Chen, and X. Hu, "S2GAE: Self-Supervised Graph Autoencoders are Generalizable Learners with Graph Masking," in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, ser. WSDM '23, 2023, pp. 787–795. [Online]. Available: https://doi.org/10.1145/3539597.3570404

[89] Z. Tan, R. Guo, K. Ding, and H. Liu, "Virtual Node Tuning for Few-shot Node Classification," in *KDD*, 2023, pp. 2177–2188.

[90] S. Thakoor, C. Tallec, M. G. Azar, R. Munos, P. Veličković, and M. Valko, "Bootstrapped representation learning on graphs," in *ICLR*, 2021.

[91] Y. Tian, H. Song, Z. Wang, H. Wang, Z. Hu, F. Wang, N. V. Chawla, and P. Xu, "Graph Neural Prompting with Large Language Models," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[92] M. Tsimpoukelli, J. L. Menick, S. Cabi, S. M. A. Eslami, O. Vinyals, and F. Hill, "Multimodal Few-Shot Learning with Frozen Language Models," in *NeurIPS*, vol. 34, 2021, pp. 200–212.

[93] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *ICLR*, 2018.

[94] P. Velickovic, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep Graph Infomax," in *ICLR*, 2019.

[95] C. Wang, S. Pan, G. Long, X. Zhu, and J. Jiang, "MGAE: Marginalized Graph Autoencoder for Graph Clustering," in *CIKM*, 2017, pp. 889–898.

[96] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu *et al.*, "Scientific discovery in the age of artificial intelligence," *Nature*, vol. 620, no. 7972, pp. 47–60, 2023.

[97] J. Wang, D. Chen, C. Luo, X. Dai, L. Yuan, Z. Wu, and Y.-G. Jiang, "ChatVideo: A Tracklet-centric Multimodal and Versatile Video Understanding System," *arXiv preprint*, 2023.

[98] P. Wang, K. Agarwal, C. Ham, S. Choudhury, and C. K. Reddy, "Self-Supervised Learning of Contextual Embeddings for Link Prediction in Heterogeneous Networks," in *The Web Conference*, 2021, pp. 2946–2957.

[99] R. Wang, Y. Li, S. Lin, W. Wu, H. Xie, Y. Xu, and J. C. Lui, "Common neighbors matter: fast random walk sampling with common neighbor awareness," *TKDE*, vol. 35, no. 5, pp. 4570–4584, 2022.

[100] X. Wang, Y. Zhang, and C. Shi, "Hyperbolic heterogeneous information network embedding," in *AAAI*, vol. 33, 2019, pp. 5337–5344.

[101] X. Wang, N. Liu, H. Han, and C. Shi, "Self-supervised Heterogeneous Graph Neural Network with Co-contrastive Learning," in *KDD*, 2021, pp. 1726–1736.

[102] Y. Wang, N. Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr, "Knowledge Graph Prompting for Multi-Document Question Answering," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[103] Z. Wang, Z. Liu, T. Ma, J. Li, Z. Zhang, X. Fu, Y. Li, Z. Yuan, W. Song, Y. Ma, Q. Zeng, X. Chen, J. Zhao, J. Li, M. Jiang, P. Lio, N. Chawla, C. Zhang, and Y. Ye, "Graph foundation models: A comprehensive survey," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2505.15116v1

[104] Z. Wen and Y. Fang, "Augmenting Low-Resource Text Classification with Graph-Grounded Pre-Training and Prompting," in *SIGIR*, 2023, pp. 506–516.

[105] ——, "Prompt Tuning on Graph-augmented Low-resource Text Classification," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[106] Z. Wen, Y. Fang, Y. Liu, Y. Guo, and S. Hao, "Voucher Abuse Detection with Prompt-based Fine-tuning on Graph Neural Networks," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023.

[107] J. Wu, X. Wang, F. Feng, X. He, L. Chen, J. Lian, and X. Xie, "Self-supervised Graph Learning for Recommendation," in *SIGIR*, 2021.

[108] J. Wu, S. Li, A. Deng, M. Xiong, and H. Bryan, "Prompt-and-Align: Prompt-Based Social Alignment for Few-Shot Fake News Detection," in *CIKM*, 2023.

[109] X. Wu, K. Zhou, M. Sun, X. Wang, and N. Liu, "A Survey of Graph Prompting Methods: Techniques,

Applications, and Challenges," *arXiv preprint*, 2023.
[110] Y. Wu, R. Xie, Y. Zhu, F. Zhuang, X. Zhang, L. Lin, and Q. He, "Personalized Prompt for Sequential Recommendation," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
[111] Y. Xie, Z. Xu, and S. Ji, "Self-Supervised Representation Learning via Latent Graph Prediction," in *ICML*, 2022, pp. 24 460–24 477.
[112] Y. Xie, Z. Xu, J. Zhang, Z. Wang, and S. Ji, "Self-Supervised Learning of Graph Neural Networks: A Unified Review," *TPAML*, vol. 45, no. 2, pp. 2412–2429, 2023.
[113] D. Xu, W. Cheng, D. Luo, H. Chen, and X. Zhang, "InfoGCL: Information-Aware Graph Contrastive Learning," in *NeurIPS*, vol. 34, 2021, pp. 30 414–30 425.
[114] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *ICLR*, 2018.
[115] Y. Xu, X. Liu, K. Duan, Y. Fang, Y.-N. Chuang, D. Zha, and Q. Tan, "Graphfm: A comprehensive benchmark for graph foundation model," *arXiv preprint*, 2024. [Online]. Available: https://arxiv.org/abs/2406.08310v2
[116] C. Yang, D. Bo, J. Liu, Y. Peng, B. Chen, H. Dai *et al.*, "Data-centric graph learning: A survey," *IEEE Transactions on Big Data*, 2025.
[117] H. Yang, X. Zhao, Y. Li, H. Chen, and G. Xu, "An Empirical Study Towards Prompt-Tuning for Graph Contrastive Pre-Training in Recommendations," in *NeurIPS*, 2023.
[118] Z. Yi, I. Ounis, and C. Macdonald, "Contrastive Graph Prompt-tuning for Cross-domain Recommendation," *TOIS*, 2023.
[119] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *NeurIPS*, 2020, pp. 5812–5823.
[120] J. Yu, H. Yin, X. Xia, T. Chen, L. Cui, and N. Q. V. Hung, "Are Graph Augmentations Necessary? Simple Graph Contrastive Learning for Recommendation," in *SIGIR*, 2022.
[121] J. Yu, H. Yin, X. Xia, T. Chen, J. Li, and Z. Huang, "Self-Supervised Learning for Recommender Systems: A Survey," *TKDE*, pp. 1–20, 2023.
[122] X. Yu, Z. Gong, C. Zhou, Y. Fang, and H. Zhang, "Samgpt: Text-free graph foundation model for multi-domain pre-training and cross-domain adaptation," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2502.05424v2
[123] X. Yu, S. Ye, R. Liang, C. Zhou, H. Cheng, X. Zhang, and Y. Fang, "Evaluating progress in graph foundation models: A comprehensive benchmark and new insights," *arXiv preprint*, 2026. [Online]. Available: https://arxiv.org/abs/2603.10033v1
[124] H. Zhang, X. Li, and L. Bing, "Video-LLaMA: An Instruction-tuned Audio-Visual Language Model for Video Understanding," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023.
[125] J. Zhang, H. Zhang, C. Xia, and L. Sun, "Graph-Bert: Only Attention is Needed for Learning Graph Representations," *arXiv preprint*, 2020.
[126] J. Zhang, Y. Dong, Y. Wang, J. Tang, and M. Ding, "ProNE: Fast and Scalable Network Representation Learning," in *IJCAI*, 2019, pp. 4278–4284.
[127] M. Zhang and Y. Chen, "Link Prediction Based on Graph Neural Networks," in *NeurIPS*, 2018.
[128] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto, "Benchmarking Large Language Models for News Summarization," *Transactions of the Association for Computational Linguistics*, 2024.
[129] W. Zhang, Y. Zhu, M. Chen, Y. Geng, Y. Huang, Y. Xu, W. Song, and H. Chen, "Structure Pretraining and Prompt Tuning for Knowledge Graph Transfer," in *The Web Conference*, 2023, pp. 2581–2590.
[130] Z. Zhang, H. Li, Z. Zhang, Y. Qin, X. Wang, and W. Zhu, "Large graph models: A perspective," *arXiv preprint*, 2023.
[131] H. Zhao, A. Chen, X. Sun, H. Cheng, and J. Li, "All in one and one for all: A simple yet effective method towards cross-domain graph pretraining," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024.
[132] H. Zhao, S. Liu, C. Ma, H. Xu, J. Fu, Z.-H. Deng, L. Kong, and Q. Liu, "GIMLET: A Unified Graph-Text Model for Instruction-Based Molecule Zero-Shot Learning," in *Advances in Neural Information Processing Systems 36*, 2023.
[133] W. Zhao, Q. Wu, C. Yang, and J. Yan, "GraphGLOW: Universal and Generalizable Structure Learning for Graph Neural Networks," in *KDD*, 2023, pp. 3525–3536.
[134] Z. Zhao, Y. Su, Y. Li, Y. Zou, R. Li, and R. Zhang, "A survey on self-supervised graph foundation models: Knowledge-based perspective," *arXiv preprint*, 2024. [Online]. Available: https://arxiv.org/abs/2403.16137v3
[135] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph Contrastive Learning with Adaptive Augmentation," in *The Web Conference*, 2021, pp. 2069–2080.
[136] Y. Zhu, J. Guo, and S. Tang, "SGL-PT: A Strong Graph Learner with Graph Prompt Tuning," *arXiv preprint*, 2023.
[137] Y. Zhu, Y. Wang, H. Shi, Z. Zhang, and S. Tang, "Graph-Control: Adding Conditional Control to Universal Graph Pre-trained Models for Graph Domain Transfer Learning," in *Proceedings of the ACM Web Conference 2024*, 2024.